

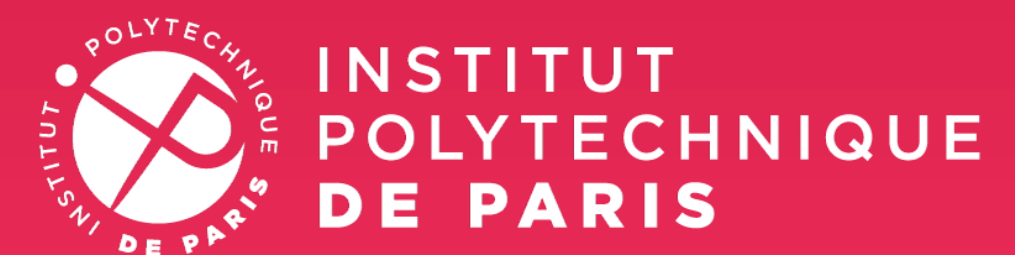
CSC_4CS02_TP

Cryptologie - Clé publique

Sébastien Canard

Weiqiang Wen (bureau 4C55,
weiqiang.wen@telecom-paris.fr)

Année 2025-2026



Cryptographie asymétrique - définitions

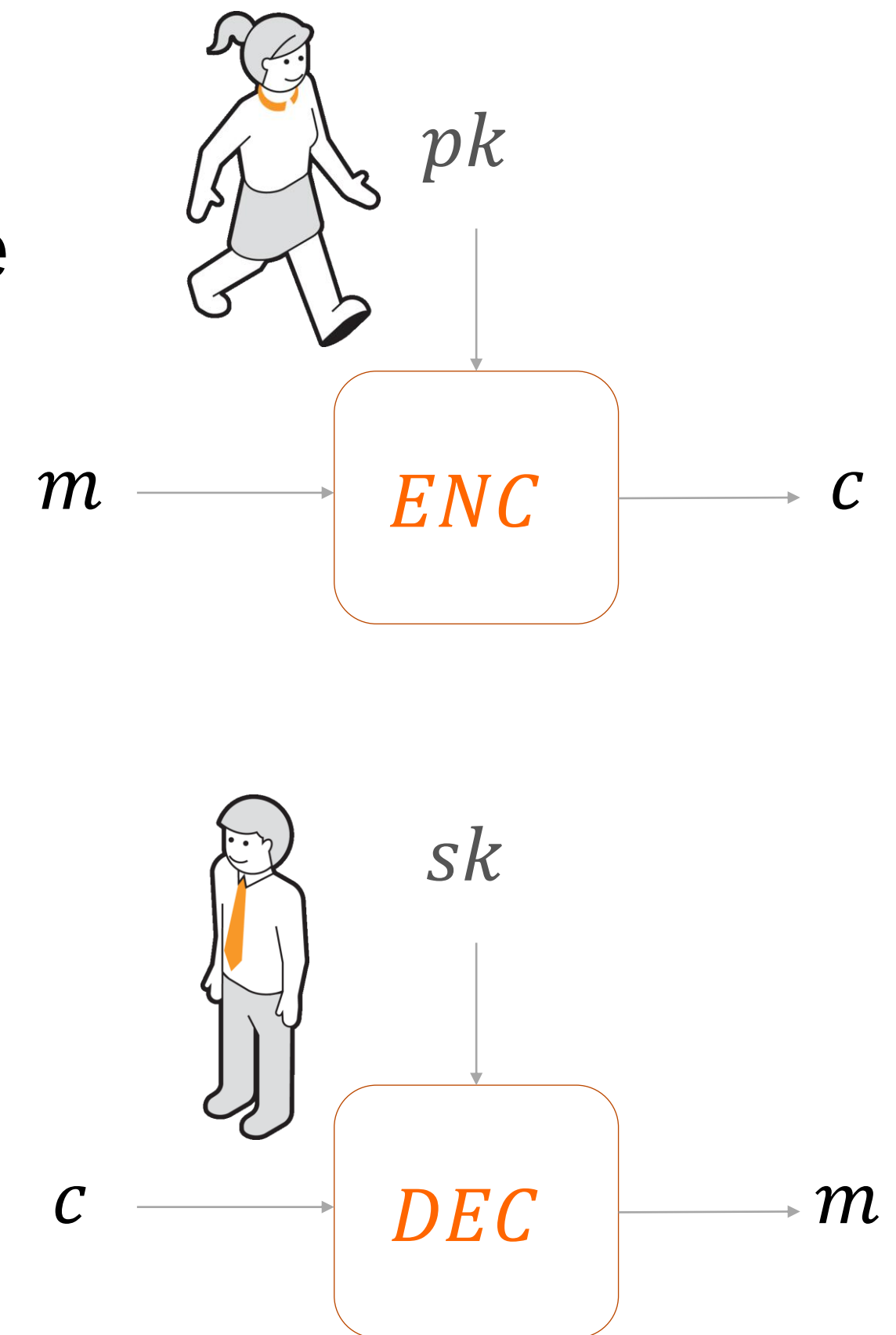
(Chiffrement à clé publique et signature)

Cryptographie à clé publique (Diffie et Hellman, 1976)

- 1^{er} question : **faut-il garder secrète la clé de chiffrement ?**
- Diffie et Hellman (1976) : pourquoi ne pas rendre la clé de chiffrement publique ?
- Cryptographie à clé publique
 - Une clé est rendue publique et peut être utilisée par n'importe qui. Elle est utilisée pour l'étape non sensible
 - Une clé reste secrète (ou privée). Elle doit être utilisée pour l'étape sensible
- Quelques exemples que nous verrons : RSA, ElGamal, RSA-PSS, Schnorr...

Chiffrement à clé publique

- Génération des clés
 - Pour un paramètre de sécurité donné
 - Génération d'une clé privée sk et de la clé publique pk associée
 - sk est gardée secrète, et pk est diffusée largement
- Chiffrement Enc
 - C'est l'étape non sensible
 - Utilisation de la clé publique pk et d'un message m à protéger
 - Donne en sortie un chiffré c
 - Cette étape peut (doit ?) être probabiliste
- Déchiffrement Dec
 - C'est l'étape sensible
 - Utilisation de la clé privée sk et du chiffré c
 - Donne en sortie le message clair m



Sécurité d'un système de chiffrement à clé publique

- Sens unique (OW)
 - Propriété de base que l'on cherche à obtenir
 - A partir de c et de pk , il doit être infaisable de trouver m
- Indistinguabilité (IND)
 - Mais si je peux deviner quel message peut être chiffré, ce n'est pas suffisant
 - Connaissant/choisissant m_0 et m_1 et un chiffré c de m_b ($b \in_R \{0,1\}$), décider si $b = 0$ ou 1 (c'est-à-dire il est infaisable même-si récupérer un bit de m .)
- Non malléabilité (NM)
 - Peut-on modifier le message chiffré sans connaître le clair ?
 - A partir du chiffré c d'un message m inconnu, calculer le chiffré d'un message proche de m (exemple $m + 1$)
- Attaques possibles
 - CPA : attaque à texte clair choisi
 - CCA : attaque à texte chiffré choisi (niveau 1 ou 2)

Sécurité d'un système de chiffrement à clé publique

IND-CPA

Adversaire

l'accès à O^{Enc} avant et après avoir vu C^*

$O^{Enc}(\{M_i\}_i)$

$\{C_i\}_i$

(m_0, m_1, C^*)

Challenger

$b \leftarrow_R \{0,1\}$

Choisit $m_0 \neq m_1$

$C^* = Enc(m_b)$

b^*

Si $b^* = b$: Adv réussit

Le schéma est sûr au sens IND-CPA si $\Pr[\text{Adv réussit}] = \text{negl}(\lambda)$.

Sécurité d'un système de chiffrement à clé publique

IND-CCA-1

Adversaire

l'accès à O^{Enc} avant et après avoir vu C^*

$O^{Enc}(\{M_i\}_i)$

$\{C_i\}_i$

$O^{Dec}(\{C'_i\}_i)$

$\{M'_i\}_i$

(m_0, m_1, C^*)

Challenger

$b \leftarrow_R \{0,1\}$

Choisit $m_0 \neq m_1$

$C^* = Enc(m_b)$

b^*

Si $b^* = b$: Adv réussit

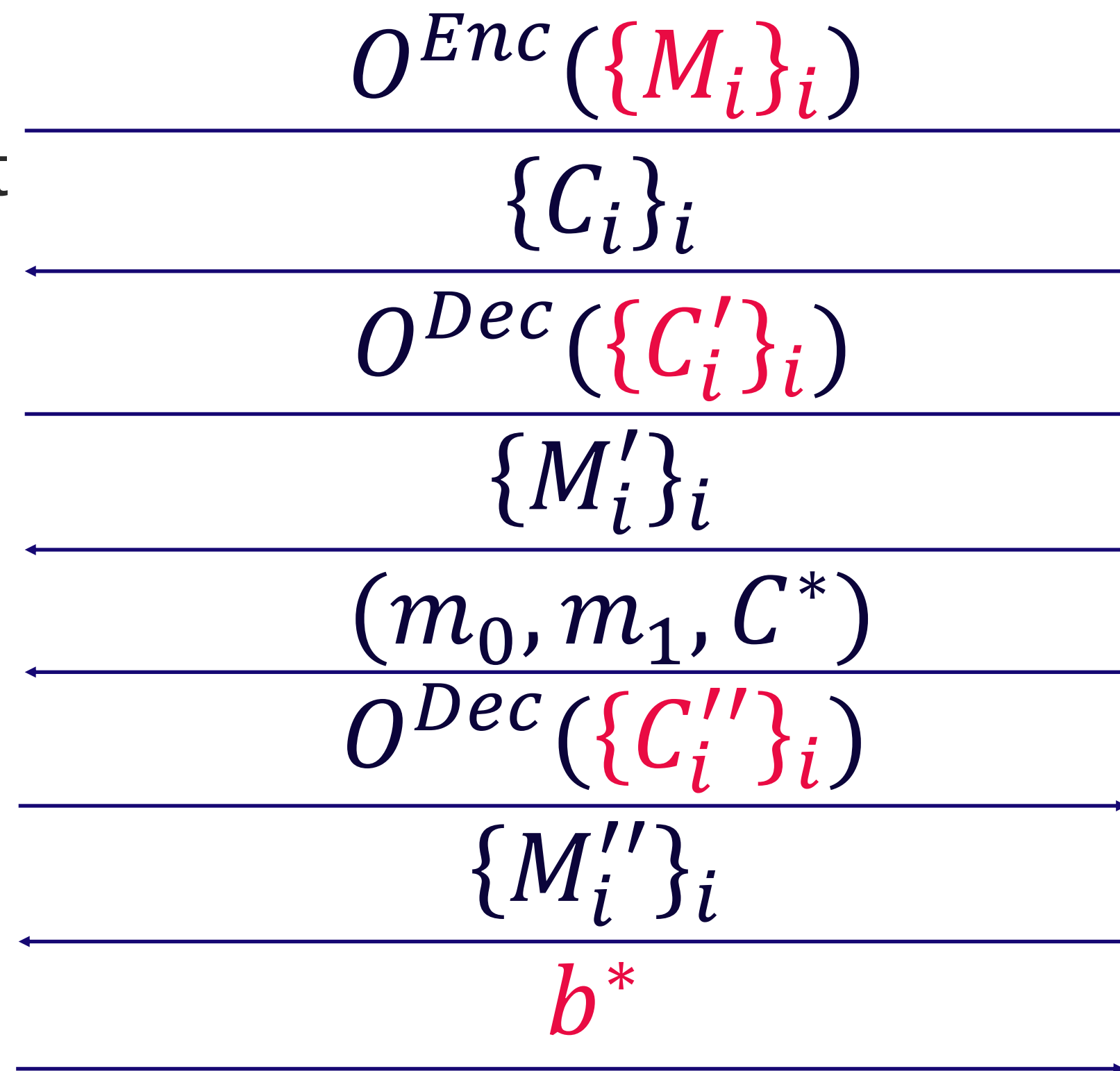
Le schéma est sûr au sens IND-CCA-1 si $\Pr[\text{Adv réussit}] = \text{negl}(\lambda)$.

Sécurité d'un système de chiffrement à clé publique

IND-CCA-2

Adversaire

l'accès à O^{Enc} avant et après avoir vu C^*



Challenger

$b \leftarrow_R \{0,1\}$
Choisit $m_0 \neq m_1$
 $C^* = Enc(m_b)$

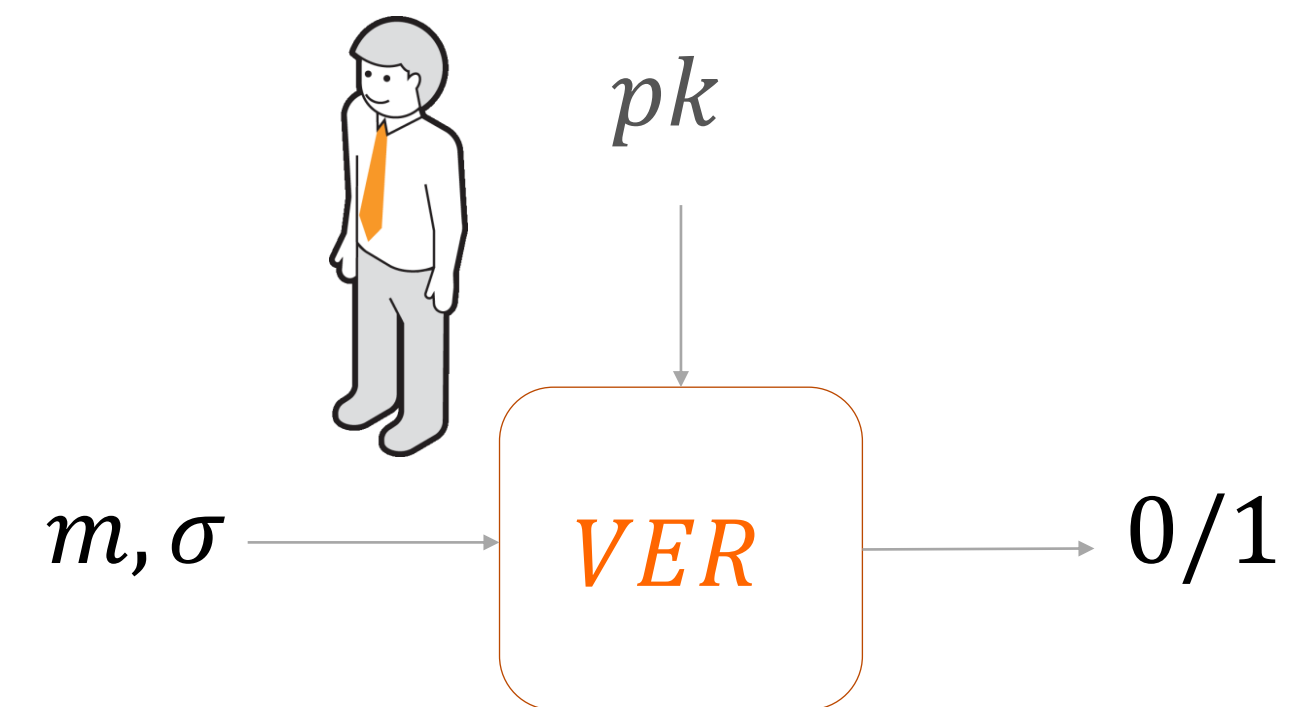
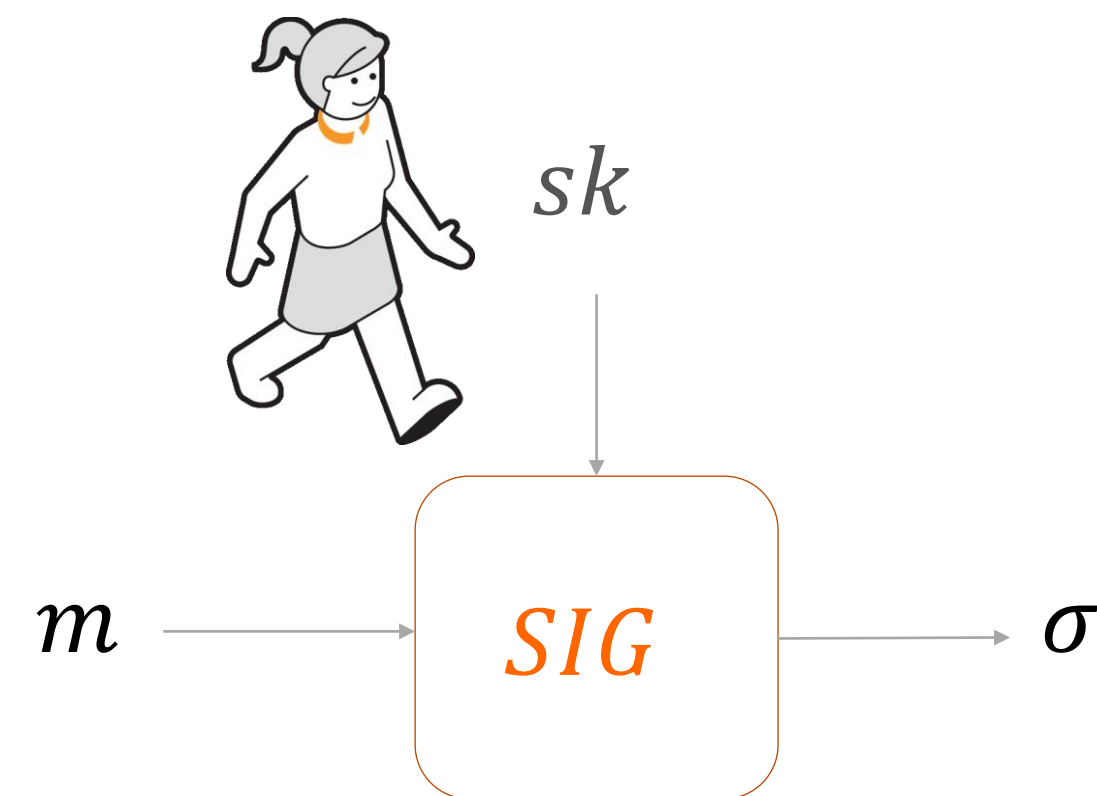
$C^* \notin \{C''_i\}_i$

Si $b^* = b$: Adv réussit

Le schéma est sûr au sens IND-CCA-2 si $\Pr[\text{Adv réussit}] = \text{negl}(\lambda)$.

Signature numérique

- Le fait qu'une seule entité connaisse la clé secrète, et que l'autre devienne publique, permet de réaliser une véritable signature numérique
 - La signature devient publiquement vérifiable
 - Nous pouvons obtenir la non-répudiation



Signature numérique

- Génération des clés
 - Pour un paramètre de sécurité donné
 - Génération d'une clé privée sk et de la clé publique pk associée
 - sk est gardée secrète, et pk est diffusée largement
- Signature Sig
 - C'est l'étape sensible
 - Utilisation de la clé privée sk et d'un message m à signer
 - Donne en sortie une signature σ
 - Cette étape peut être probabiliste
- Vérification Ver
 - C'est l'étape non sensible (n'importe qui peut vérifier une signature)
 - Utilisation de la clé publique pk , du message m et de la signature σ
 - Donne en sortie un bit 0 (signature non valide) ou 1 (signature valide)

Sécurité d'un système de signature numérique

- Objectifs de l'adversaire
 - Falsification : être capable de sortir un message m et une signature σ valide
- Connaissances possibles de l'attaquant
 - Uniquement la clé publique
 - Accès supplémentaire à un oracle de signature

Adversaire

$O^{Sign}(\{M_i\}_i)$

$\{\sigma_i\}_i$

(M^*, σ^*)

Challenger

$M^* \notin \{M_i\}_i$

Si σ^* est valide sur M^* ,
Adv réussit

Le schéma est sûr au sens UF-CMA si $\Pr[\text{Adv réussit}] = \text{negl}(\lambda)$.

Cryptographie à clé publique / à clé secrète

- Systèmes symétriques / à clé secrète
 - Une clé secrète pour chaque paire d'entités
 - Même clé pour chiffrer et déchiffrer
 - Même clé pour signer et vérifier

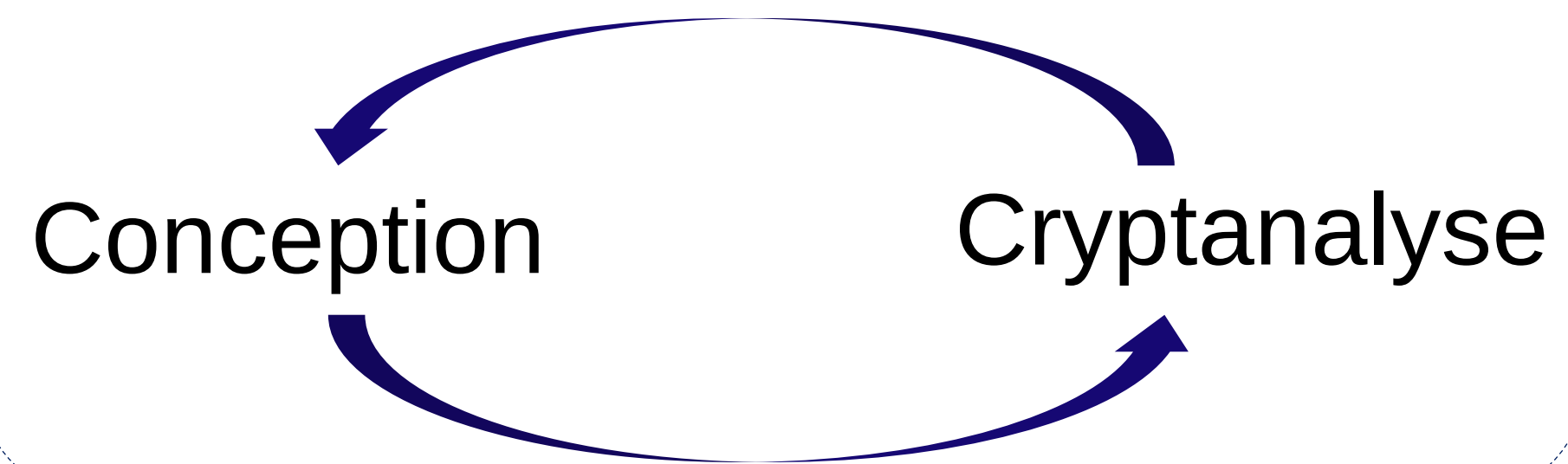
- **Plus rapide**
- 1 clé partagée par chaque couple de personne
- Pas de service de non répudiation

- Systèmes asymétriques / à clé publique
 - Une paire de clés (l'une publique, l'autre privée) pour chaque entité
 - Clé publique pour chiffrer / Clé privée pour déchiffrer
 - Clé privée pour signer / Clé publique pour vérifier

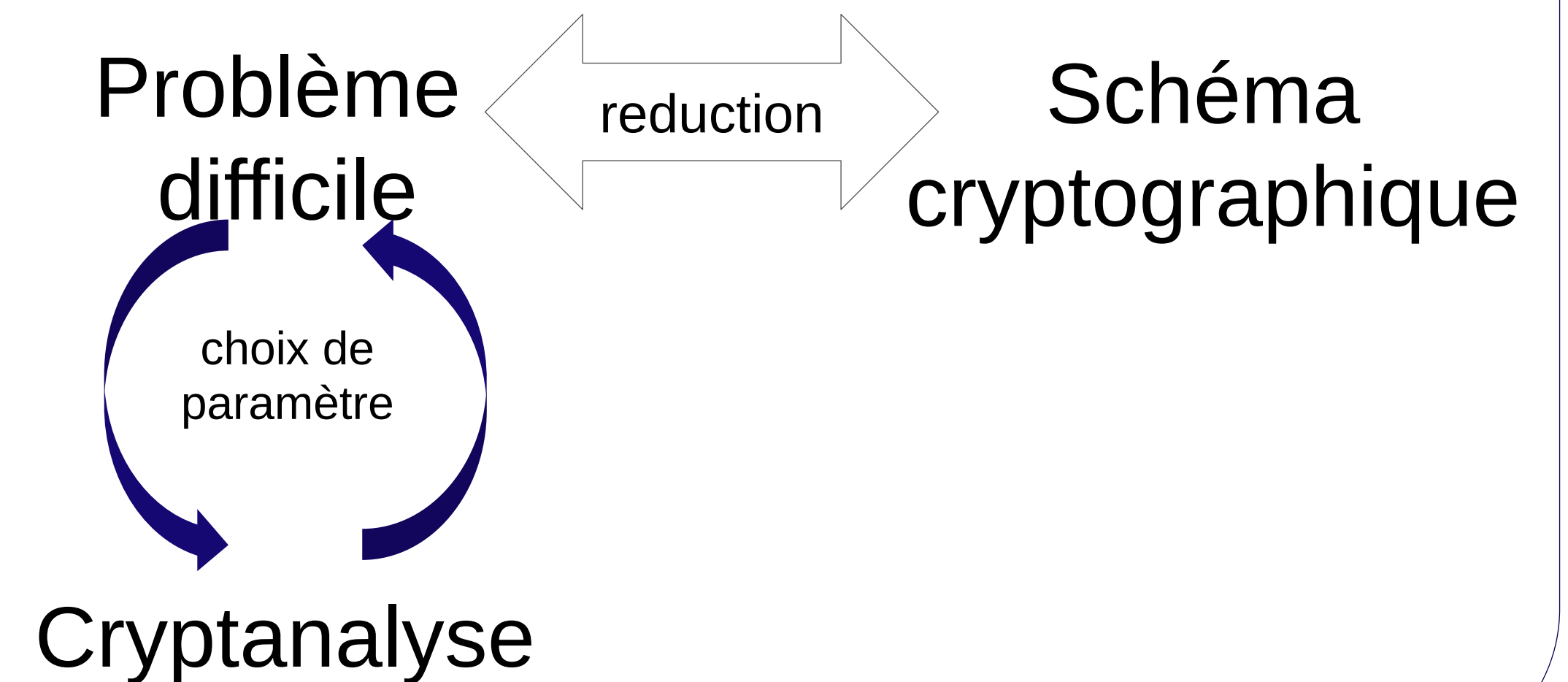
- Moins rapide
- Plus simple : **1 paire de clés par personne**
- Service de **non répudiation**

Deux types de construction

Cryptographie à clé secrète



Cryptographie à clé publique



Attardons-nous un peu sur la notion de **problème difficile**

Notion d'algorithme

- Algorithme
 - Suite finie d'opérations élémentaires constituant un schéma de calcul ou de résolution d'un problème (Petit Larousse)
- En pratique : Mathématiques $\neq$ Informatique
 - Mathématiques : on résout un problème en montrant l'existence d'une solution
 - Informatique : on cherche à construire cette solution en s'intéressant à l'efficacité de la construction

Algorithme et cryptographie

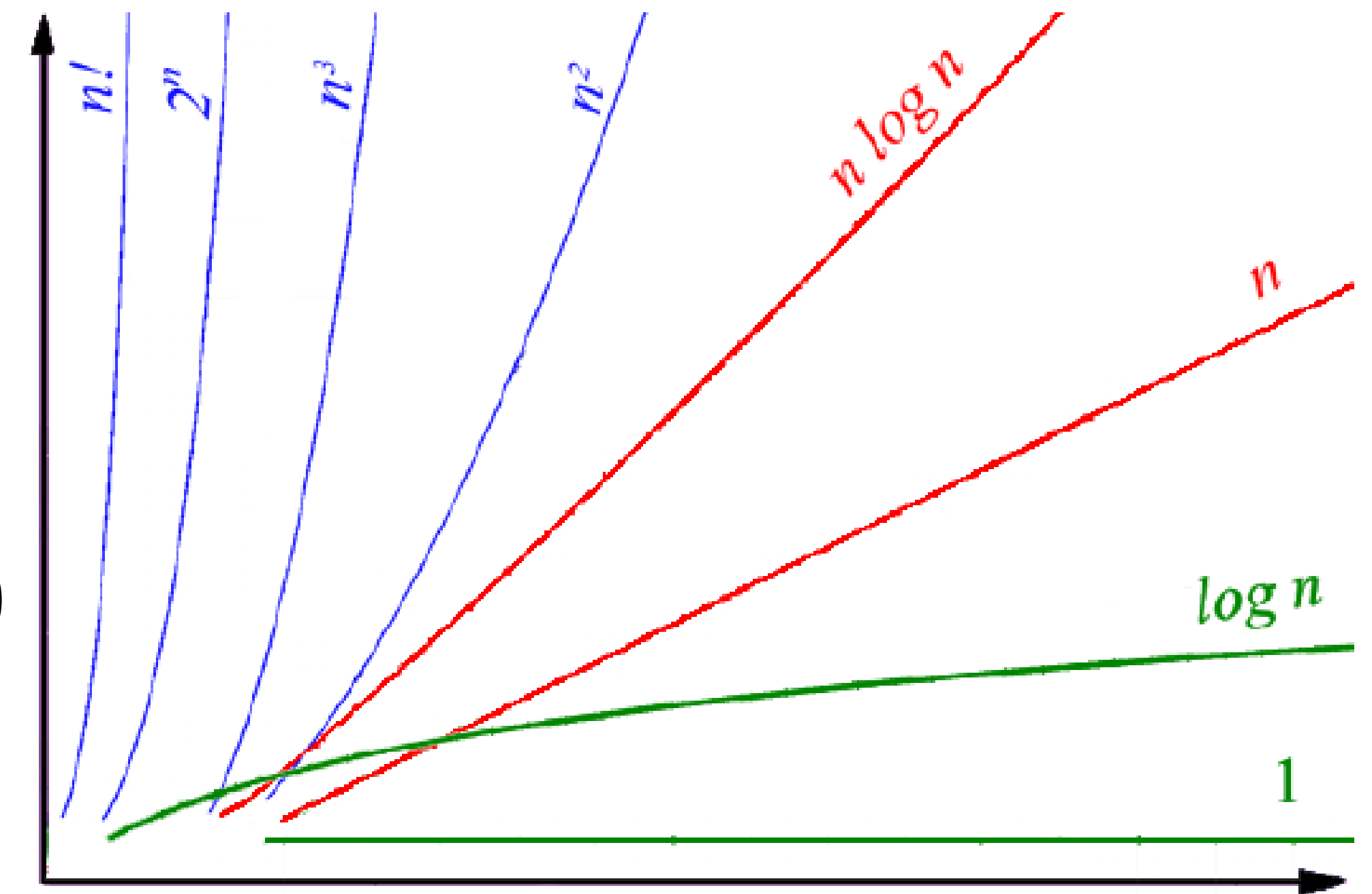
- Représenter les tâches à accomplir, manipuler des objets mathématiques
 - Calculer un chiffrement
 - Générer une signature
 - Retrouver une clé
- Mesurer un algorithme
 - La variable/le paramètre = la taille des données en entrée de l'algorithme
 - Temps = nombre d'opérations élémentaires
 - Espace = mémoire nécessaire à l'algorithme
 - Comportement = mesure asymptotique
 - Variantes : cas « moyen » ou « pire » des cas

Comparaison de fonctions (notations de Landau)

- Comparaison de deux fonctions croissantes positives $f, g : \mathbb{N} \rightarrow \mathbb{Z}$
 - $f = \mathcal{O}(g)$ si $\exists c > 0$ tel que $f(n) < c \cdot g(n)$ pour tout n
 - $f = \Omega(g)$ si $g = \mathcal{O}(f)$
 - $f = \Theta(g)$ si $f = \mathcal{O}(g)$ et $g = \mathcal{O}(f)$
 - $f = \sigma(g)$ si $\forall c > 0$, on a $f(n) < c \cdot g(n)$ pour tout n
 - $f = \omega(g)$ si $g = \sigma(f)$
- Exemples
 - $3n^2 + 3n + 1 = \Omega(9n^2) = \mathcal{O}(n^3/100)$
 - $1000n^{512} = \mathcal{O}(1,01^n)$
 - $2^n = \mathcal{O}(10^n) = \mathcal{O}(n!)$
 - $\log n = \sigma(n) = \sigma(n \log n) = \mathcal{O}(n \log n)$

Hiérarchie entre fonctions

- Hiérarchie élémentaire
 - f est logarithmique si $f(n) = \mathcal{O}(\log n^{\mathcal{O}(1)})$
 - f est polynomiale si $f(n) = \mathcal{O}(n^{\mathcal{O}(1)})$
 - f est exponentielle si $f(n) = \Omega(e^{\Omega(n)})$
- Hiérarchie intermédiaire
 - Pire que polynomiale ...
 - f est super-polynomiale si $f(n) = \Omega(n^{\omega(1)})$
 - f est sous-exponentielle si $f(n) = \mathcal{O}(e^{\sigma(n)})$
 - ... moins pire qu'exponentielle
- Ex. la fonction $L_n[\alpha, c] = e^{c \cdot n^\alpha \cdot (\log n)^{1-\alpha}}$
 - Polynomiale si $\alpha = 0$
 - Exponentielle si $\alpha = 1$
 - Sous-exponentielle et super-polynomiale sinon



Efficacité en pratique

- Logarithmique : toujours facile
 - Polynomial : facile (si l'exposant reste petit)
 - Sous-exponentiel, super-polynomial : très difficile, probablement faisable jusqu'à une taille critique
 - Exponentiel : extrêmement difficile, très vite impossible en pratique
-
- **Faisable = polynomial**
 - Pour être *utilisable*, un algorithme doit être
 - Polynomial en moyenne
 - Implémentable facilement
 - Efficace en pratique

Exemples de complexités

$n = N = \log_2 N$	64	128	512	1024
n^2	2^{12}	2^{14}	2^{18}	2^{20}
n^3	2^{18}	2^{21}	2^{27}	2^{30}
$\mathcal{O}\left(e^{n^{1/3}(\log n)^{2/3}}\right)$	2^{30}	2^{41}	2^{78}	2^{105}
$2^{n/2} = \sqrt{N}$	2^{32}	2^{64}	2^{256}	2^{512}
$2^n = N$	2^{64}	2^{128}	2^{512}	2^{1024}

Rappel mathématique (groupe, anneau et corps)

Je vous laisse revoir cette partie, qui sera rapidement passée au cours.

Arithmétique modulaire

- Référence à une l'horloge

- Heures :

- $2h + 5h = 7h$
 - $9h + 4h = 1h \pmod{12h}$
 - $8h + 12h = 8h \pmod{12h}$

- Minutes :

- $45 + 25 = 10 \pmod{60 \text{ min}}$
 - $50 + 35 = 20 \pmod{60 \text{ min}}$

Domaine euclidien :

si $a > b$ alors on a $a = b \cdot q + r$ pour unique r t.q. $0 \leq r < b$ (module q)

=> on déduit $a = r \pmod{q}$

- Additions modulaires dans $\mathbb{Z}_7$

- $\mathbb{Z}_7 = \{0,1,2,3,4,5,6\}$
 - $2 + 4 = 6 \pmod{7}$
 - $4 + 5 = 9 = 7 + 2 = 2 \pmod{7}$
 - $3 + 4 = 7 = 0 \pmod{7}$

+	0	1	2	3	4	5	6
0	0	1	2	3	4	5	6
1	1	2	3	4	5	6	0
2	2	3	4	5	6	0	1
3	3	4	5	6	0	1	2
4	4	5	6	0	1	2	3
5	5	6	0	1	2	3	4
6	6	0	1	2	3	4	5

Groupe additif $(\mathbb{Z}_N, +)$

- $(\mathbb{Z}_N, +)$ forme un groupe commutatif d'ordre N
- Définition d'un groupe
 - Un groupe est un couple ensemble-loi de composition $(\mathbb{G}, *)$ qui vérifie
 - associativité : $\forall a, b, c \in \mathbb{G}, (a * b) * c = a * (b * c)$
 - élément neutre : $\exists e \in \mathbb{G}, \forall a \in \mathbb{G}, a * e = e * a = a$
 - inversion : $\forall a \in \mathbb{G}, \exists b \in \mathbb{G}, a * b = b * a = e$
- Propriétés
 - Commutativité : $\forall a, b \in \mathbb{G}, a * b = b * a$
 - Ordre de $\mathbb{G}$: nombre d'éléments du groupe $\mathbb{G}$

Multiplication modulaire dans $\mathbb{Z}_7$

- $\mathbb{Z}_7 = \{0,1,2,3,4,5,6\}$
- $3 \times 5 = 5 + 5 + 5 = 10 + 5 = 3 + 5 = 8 = 7 + 1 = 1 \pmod{7}$
- $3 \times 5 = 15 = 2 \times 7 + 1 = 1 \pmod{7}$
- $6 \times 4 = 24 = 3 \times 7 + 3 = 3 \pmod{7}$

$\times$	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6
2	0	2	4	6	1	3	5
3	0	3	6	2	5	1	4
4	0	4	1	5	2	6	3
5	0	5	3	1	6	4	2
6	0	6	5	4	3	2	1

Multiplications modulaires dans $\mathbb{Z}_7$ et $\mathbb{Z}_8$

×	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6
2	0	2	4	6	1	3	5
3	0	3	6	2	5	1	4
4	0	4	1	5	2	6	3
5	0	5	3	1	6	4	2
6	0	6	5	4	3	2	1

×	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7
2	0	2	4	6	0	2	4	6
3	0	3	6	1	4	7	2	5
4	0	4	0	4	0	4	0	4
5	0	5	2	7	4	1	6	3
6	0	6	4	2	0	6	4	2
7	0	7	6	5	4	3	2	1

Anneau $(\mathbb{Z}_N, +, \cdot)$

- $(\mathbb{Z}_N, +, \cdot)$ forme un anneau commutatif
- Définition d'anneau
 - Un anneau est un triplet, ensemble et deux lois de composition, $(\mathbb{G}, +, \cdot)$ qui vérifie
 - $(\mathbb{G}, +)$ est un groupe commutatif avec élément neutre 0
 - associativité : $\forall a, b, c \in \mathbb{G}, (a \cdot b) \cdot c = a \cdot (b \cdot c)$
 - élément neutre : $\exists e \in \mathbb{G}, \forall a \in \mathbb{G}, a \cdot e = e \cdot a = a$
 - distributivité : $\forall a, b, c \in \mathbb{G}, a \cdot (b + c) = a \cdot b + a \cdot c$
- Propriétés
 - Anneau commutatif : $\forall a, b \in \mathbb{G}, a \cdot b = b \cdot a$
 - Élément inversible : $a \neq 0$ est inversible par rapport à la loi $\cdot$ si $\exists b \in \mathbb{G}, a \cdot b = b \cdot a = e$

Inverse modulaire dans $\mathbb{Z}_7$

- Inverse de a modulo N
 - C'est l'entier $b = a^{-1}$ tel que $a \times b = 1 \pmod{N}$
- $\mathbb{Z}_7^* = \{1, 2, 3, 4, 5, 6\}$
- $3^{-1} = 5 \pmod{7}$
- $4^{-1} = 2 \pmod{7}$
- $6^{-1} = 6 \pmod{7}$

×	1	2	3	4	5	6
1	1	2	3	4	5	6
2	2	4	6	1	3	5
3	3	6	2	5	1	4
4	4	1	5	2	6	3
5	5	3	1	6	4	2
6	6	5	4	3	2	1

Inverse modulaire dans $\mathbb{Z}_8$

- 2, 4 et 6 sont non inversibles
- 1, 3, 5 et 7 sont inversibles (et leurs propres inverses)
- Attention aux écritures
 - Eviter $3 = \frac{1}{3}$
 - Proscrire $\sqrt{1} = 3$
 - Utiliser $3^{-1} = 3 \pmod{8}$

×	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7
2	0	2	4	6	0	2	4	6
3	0	3	6	1	4	7	2	5
4	0	4	0	4	0	4	0	4
5	0	5	2	7	4	1	6	3
6	0	6	4	2	0	6	4	2
7	0	7	6	5	4	3	2	1

Éléments inversibles

- Définition
 - $\mathbb{Z}_N^*$ = l'ensemble des éléments inversibles modulo N
 - Attention : $\mathbb{Z}_N^* \neq \mathbb{Z}_N \setminus \{0\}$
- Exemples
 - $\mathbb{Z}_7^* = \{1,2,3,4,5,6\}$: tous les éléments sont inversibles
 - $\mathbb{Z}_8^* = \{1,3,5,7\}$: on a $\mathbb{Z}_8^* \neq \mathbb{Z}_8 \setminus \{0\}$
- Groupe multiplicatif $\mathbb{Z}_N^*$
 - $(\mathbb{Z}_N^*, \cdot)$ forme un groupe multiplicatif

Critères d'inversibilité

- Question : quels sont les entiers inversibles modulo N ?
- Critère
 - $x \in \mathbb{Z}_N^*$ est **inversible** modulo N **si et seulement si** $\text{pgcd}(x, N) = 1$
 - Preuve : Théorème de Bézout
- Cas de $N = p$ premier : $\mathbb{Z}_p$
 - Tous les éléments non-nuls de $\mathbb{Z}_p$ sont forcément premiers avec p
 - Si p est premier, nous avons alors forcément

$$\mathbb{Z}_p^* = \mathbb{Z}_p \setminus \{0\}$$

- Calcul de l'inverse modulaire : $x^{-1} \pmod{N}$
 - Il existe u et v tels que $xu + Nv = \text{pgcd}(x, N) = 1$
 - Trouver l'inverse d'un élément revient à calculer u
 - **L'algorithme d'Euclide étendu** calcule des coefficients (u, v)

Division modulaire dans $\mathbb{Z}_7$

- Division modulaire = multiplication par l'inverse modulaire
- $\mathbb{Z}_7^* = \{1, 2, 3, 4, 5, 6\}$
- $5 \div 3 = 5 \times 3^{-1} = 5 \times 5 = 25 = 4 \pmod{7}$
- $4 \div 6 = 4 \times 6^{-1} = 4 \times 6 = 24 = 3 \pmod{7}$
- $6 \div 6 = 6 \times 6^{-1} = 6 \times 6 = 36 = 1 \pmod{7}$

$\div$	1	2	3	4	5	6
1	1	4	5	2	3	6
2	2	1	3	4	6	5
3	3	5	1	6	2	4
4	4	2	6	1	5	3
5	5	6	4	3	1	2
6	6	3	2	5	4	1

Corps $(\mathbb{Z}_p, +, \cdot)$

- $(\mathbb{Z}_p, +, \cdot)$ est un corps commutatif
- Définition de corps
 - Un corps est un ensemble et deux lois de composition $(\mathbb{G}, +, \cdot)$ qui vérifie
 - $(\mathbb{G}, +)$ est un groupe commutatif pour l'addition
 - $(\mathbb{G} \setminus \{0\}, \cdot)$ est un groupe pour la multiplication
 - distributivité : $\forall a, b, c \in \mathbb{G}, a \cdot (b + c) = a \cdot b + a \cdot c$
 - Corps commutatif : $\forall a, b \in \mathbb{G}, a \cdot b = b \cdot a$
- Théorème – Corps fini $\mathbb{F}_p$
 - $(\mathbb{Z}_p, +, \cdot)$ est un corps, noté $\mathbb{F}_p$, si et seulement si p est premier

Récapitulatif sur $\mathbb{Z}_n$

- Deux cas distincts
- $N = p$ est premier : $(\mathbb{Z}_p, +, \cdot)$ est un corps
 - Tous les éléments (sauf 0) sont inversibles
 - $\mathbb{Z}_p^*$ coïncide avec $\mathbb{Z}_p \setminus \{0\}$
- N est composé : $(\mathbb{Z}_N, +, \cdot)$ est un anneau
 - Seuls les éléments de $\mathbb{Z}_N^*$ sont inversibles
 - $\mathbb{Z}_N^*$ n'est pas $\mathbb{Z}_N \setminus \{0\}$

Ordre du groupe $\mathbb{Z}_N^*$

- $N = p$ premier
 - Tous les éléments non-nuls sont premiers avec p : $\mathbb{Z}_p^* = \{1, 2, 3, \dots, p-1\}$, ordre $|\mathbb{Z}_p^*| = p-1$
- N composé
 - Tous les éléments non-nuls premiers avec N sont dans $\mathbb{Z}_N^*$
 - Leur nombre est donné par la fonction d'Euler
- **Fonction d'Euler**
 - $\varphi(N)$ est le nombre d'entiers de $[1, N]$ qui sont premiers avec N
 - $\varphi(N)$ désigne l'ordre du groupe multiplicatif $\mathbb{Z}_N^*$
 - Propriétés
 - $\varphi(p) = p-1$ si p est premier
 - $\varphi(pq) = \varphi(p)\varphi(q)$
 - Formule générale de $\varphi(N)$ pour $N = \prod_i p_i^{\alpha_i}$: $\varphi(N) = \prod_{i=1}^k (p_i^{\alpha_i} - p_i^{\alpha_i-1}) = n \cdot \prod_{p|n} \left(1 - \frac{1}{p}\right)$

Exponentiation modulaire

- **Théorème de Lagrange**

- Si $\mathbb{G}$ est un groupe multiplicatif d'ordre N , alors $\forall g \in \mathbb{G}, g^N = e$
- Cas de $\mathbb{Z}_7^*$
 - $3^6 = 3 \times 3 \times 3 \times 3 \times 3 \times 3 = 9 \times 9 \times 9 = 2 \times 2 \times 2 = 8 = 1 \pmod{7}$
 - $3 \xrightarrow{\times 3} 2 \xrightarrow{\times 3} 6 \xrightarrow{\times 3} 4 \xrightarrow{\times 3} 5 \xrightarrow{\times 3} 1$ (énumérer tous les éléments dans $\mathbb{Z}_7^*$)
 - L'ordre de $|\mathbb{Z}_7^*| = \varphi(7) = 7 - 1 = 6$, d'où $3^6 = 1 \pmod{7}$

- **Petit théorème de Fermat**

- Pour p premier et tout entier a on a $a^p = a \pmod{p}$
- Preuve : $a^{\varphi(p)} = a^{p-1} = 1 \pmod{p}$, donc $a^p = a \pmod{p}$

- **Théorème d'Euler**

- Pour tout entier N et tout $a \in \mathbb{Z}_N^*$, on a $a^{\varphi(N)} = 1 \pmod{N}$

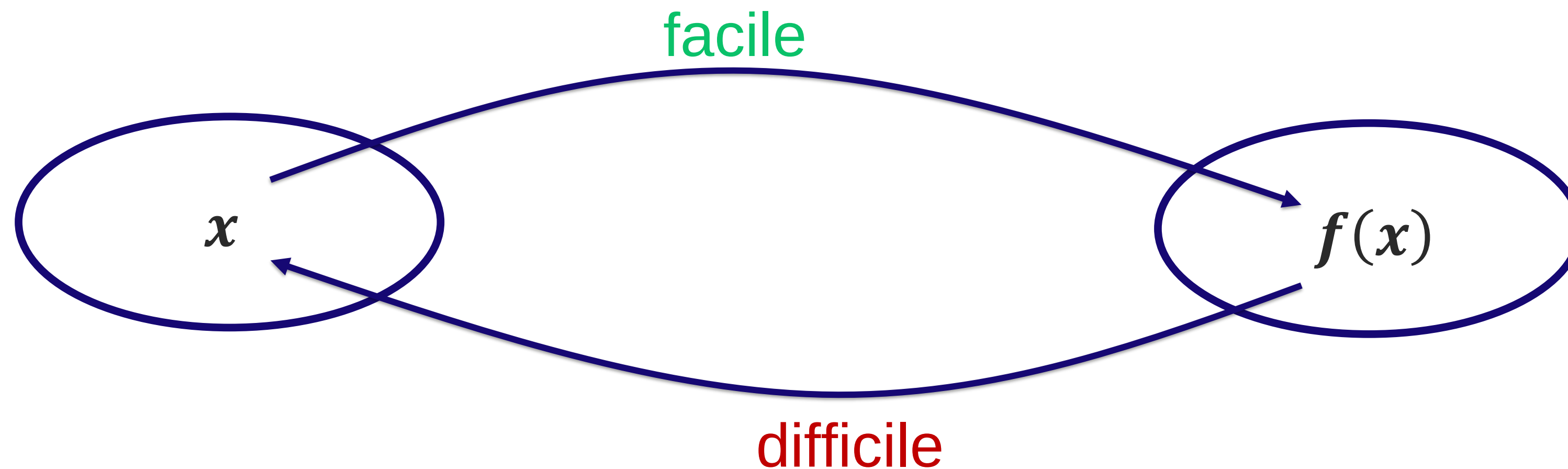
- **En pratique**

- Dans une exponentiation modulaire (modulo un entier N), les exposants doivent être pris modulo $\varphi(N)$
 - $a^p \pmod{N}$: si $p = u \times \varphi(N) + v$ pour $v < \varphi(N)$, on a $a^p = a^{u \times \varphi(N) + v} = a^v \pmod{N}$

Cryptographie asymétrique basée sur fonctions à sens unique (RSA, Elgamal, RSA-PSS, Schnorr, ECDH)

Fonction (publique) à sens unique (avec trappe)

- Principe
 - Pour une relation $y = f(x)$, calculer y est facile, et retrouver x à partir de y est difficile sans une « trappe »



- Comment construire de telles fonctions ?

Factorisation et chiffrement RSA

Factorisation des entiers

- Problème (N est exponentiel en $n = \log N$)
 - $(p, q) \rightarrow p \times q$ est facile
 - $N = p \times q \rightarrow (p, q)$ est difficile
- Algorithmes de factorisation
 - Facile si $\varphi(N)$ est connu, car on connaît $p \times q = N$ et $p + q = N - \varphi(N) + 1$ (rappel : $\varphi(N) = (p - 1)(q - 1)$)
 - Divisions successives $\Rightarrow \mathcal{O}(\sqrt{N})$
 - Algorithme de Fermat : trouver $N = a^2 - b^2 \Rightarrow \mathcal{O}(N^{1/3}) = \mathcal{O}(2^{n/3})$ (cas facile : $p - q$ petit)
 - Algorithme $p - 1$ de Pollard $\Rightarrow \mathcal{O}(p \log p)$ (cas facile : $\underline{p - 1}$ et $\underline{q - 1}$ ont des petits facteurs premiers)
 - Pollard's rho $\Rightarrow \mathcal{O}(\sqrt[4]{N})$
 - Algorithme Williams (cas facile : $p + 1$ et $q + 1$ ont des petits facteurs premiers)
 - Algorithmes sous-exponentiels : crible quadratique, courbes elliptiques, crible algébrique ($e^{\mathcal{O}((\ln N)^{1/3}(\ln \ln N)^{2/3})}$)

Factorisation – Algorithme de Fermat

1. Transformer $N = pq$ en forme $N = a^2 - b^2$

$$1. N = pq = a^2 - b^2 = (a + b)(a - b)$$

$$2. p = a + b; q = a - b$$

$$3. a = \frac{p+q}{2}; b = \frac{p-q}{2}$$

$$2. N = \left(\frac{p+q}{2}\right)^2 - \left(\frac{p-q}{2}\right)^2$$

3. Pour a de $\sqrt{N} \rightarrow \frac{p+q}{2}$:

1. On teste si $a^2 - N$ est un carré

Analyse :

$$\frac{p+q}{2} - \sqrt{N} = \frac{(\sqrt{q} - \sqrt{p})^2}{2} = \frac{(q + p - 2\sqrt{N})p}{2p} = \frac{N + p^2 - 2\sqrt{N}p}{2p} = \frac{(\sqrt{N} - p)^2}{2p}$$

Deviner $\frac{p}{q}$ par $\frac{\alpha}{\beta}$, appliquer l'algorithme de Fermat sur $N\alpha\beta = (\alpha q)(\beta p)$.

Factorisation – $p-1$ de Pollard méthode

Condition: $p - 1 = \prod_{i=1}^B q'_i$ avec certain petit nombre premier q'_i

1. Soit q_i le i -ème nombre premier à partir de 2
2. On construire $v = \prod_{i=1}^{\hat{B}} q_i^{\alpha_i}$ avec petit nombre α_i
3. Calculer $\text{pgcd}(2^v - 1, N)$

Analyse :

$$2^v = 2^{\prod_{i=1}^{\hat{B}} q_i^{\alpha_i}} \xrightarrow{\text{Probablement égal à}} 2^{k(p-1)} = 1[p]$$

Si oui, on a $p \mid 2^{k(p-1)} - 1$, donc $\text{pgcd}(2^v - 1, N) = p$

Factorisation – Pollard rho méthode

Soit $N = p \times q$ t.q. $p \leq \sqrt{N}$

1. On conçoit une fonction f , e.g., $f(x) = x^2 + 1 [N]$
2. On calcul une liste de nombre a partir de quelque nombre a_1 (e.g., $a_1 = 2$) :
 $a_1, a_2, \dots, a_k$ jusqu'à il existe i et j t.q. $a_i = a_j [p]$ (l'algorithme de Floyd pour chercher)
3. On peut simplement tester si $\text{pgcd}(a_i - a_j) > 1$

Complexité :

Il s'agit de chercher une collision modulo p , par rapport au paradoxe d'anniversaire.

La taille de la liste est $O(\sqrt{p}) = O(\sqrt[4]{N})$.

Cryptosystème RSA (Rivest, Shamir et Adleman, 1977)

- Algorithme le plus utilisé par le monde
- De nombreuses attaques mais il résiste encore
- Basé sur le problème de la factorisation

RSA – Génération des clés

- Génération des clés (pk, sk)
 - Soit $N = p \times q$ (p et q premiers)
 - L'ordre du groupe multiplicatif $\mathbb{Z}_N^* = \varphi(N) = (p - 1)(q - 1)$
 - Soit e un entier premier avec $\varphi(N) = (p - 1)(q - 1)$
 - Soit d un entier qui satisfait $d \cdot e = 1 \pmod{\varphi(N)}$
 - Bézout $\Rightarrow d \cdot e + u \cdot \varphi(N) = 1$
- Clé publique
 - $N = pq$ module public
 - e exposant public
- Clé privée
 - $d = e^{-1} \pmod{\varphi(N)}$
 - les premiers p et q

RSA – Chiffrement/Déchiffrement

Fonction à sens-unique à trappe !

- $x \rightarrow x^e \pmod{N}$ facile
- $y = x^e \rightarrow x$ difficile, excepté avec la trappe $d = e^{-1} \pmod{\varphi(N)}$
 $(y^d = x^{ed} = x^{1-u \cdot \varphi(N)} = x^1 \cdot (x^{-u})^{\varphi(N)} = x \pmod{N})$

- Chiffrement
 - $Enc(pk = (N, e), m) = c = m^e \pmod{N}$
- Déchiffrement
 - $Dec(sk = d, c) = c^d \pmod{N}$
- Vérification
 - Théorème d'Euler
 - $c^d = (m^e)^d = m^{1-u \cdot \varphi(N)} = m^1 \cdot (m^{-u})^{\varphi(N)} = m \pmod{N}$

RSA – Exemple simplifié

- Deux petits premiers : $p = 5$ et $q = 7$
- $N = 5 \times 7 = 35$, $\varphi(N) = (5 - 1) \cdot (7 - 1) = 24$
- e et d : $ed = 1(mod\ 24)$
- $e \cdot d = 1$: Non, trop petit
- $e \cdot d = 25$: Ok, mais $e = d = 5$ et alors clé privé = clé publique
- $e \cdot d = 49$: Pareil, $e = d$
- $e \cdot d = 73$: 73 est premier, raté
- $e \cdot d = 97$: 97 est premier, raté
- $e \cdot d = 121$: 11 au carré, encore raté
- $e \cdot d = 145$: $145 = 5 \times 29$, et 5 est premier : Ok
- Clé publique = $(n = 35, e = 5)$, clé privée = $d = 29$

RSA – Génération des clés

- Génération des clés (pk, sk)
 - Soit $N = p \times q$ (p et q premiers)
 - L'ordre du groupe multiplicatif $\mathbb{Z}_N^* = \varphi(N) = (p - 1)(q - 1)$
 - Soit e un entier premier avec $\varphi(N) = (p - 1)(q - 1)$
 - Soit d un entier qui satisfait $d \cdot e = 1 \pmod{\varphi(N)}$
 - Bézout $\Rightarrow d \cdot e + u \cdot \varphi(N) = 1$
- Clé publique
 - $N = pq$ module public
 - e exposant public
- Clé privée
 - $d = e^{-1} \pmod{\varphi(N)}$
 - les premiers p et q

Primalité

- Définition
 - Un nombre p est premier si ses seuls diviseurs positifs sont p et 1
 - Liste : 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, ...
- Propriétés
 - Entre n et $2n$, il y a toujours un nombre premier
 - Un nombre pair est toujours la somme de 2 premiers
 - Un nombre impair (> 5) est la somme de 3 premiers
 - Il existe une infinité de nombres premiers (Théorème d'Euclide)
- Tests de primalité
 - [Crible d'Ératosthène](#)
 - Probabilistes : [test de Fermat](#), [test de Miller-Rabin](#) ($\leq 1/4$)
 - « Réels » : [test des nombres de Mersenne](#) ([test de Lucas-Lehmer](#)), [utilisation de la factorisation de \$n - 1\$](#) , [test de la somme de Jacobi](#)
- Génération de nombres premiers
 - Recherche aléatoire, [algorithme de Maurer](#), etc.

Primalité – test de Fermat

$$\text{On teste } a^{p-1} = 1 [p] !$$

Mais ce n'est pas une condition suffisante !

$$\text{Exemple : } 2^{560} = 1 \bmod 561$$

Primalité – test de Miller-Rabin

$$a^{p-1} = 1 [p] \Rightarrow (a^{\frac{p-1}{2}} + 1)(a^{\frac{p-1}{2}} - 1) = 0[p]$$

Si p est premier, on a :

$$p \mid (a^{\frac{p-1}{2}} + 1) \text{ ou } p \mid (a^{\frac{p-1}{2}} - 1)$$

On peut continuer à tester ces deux possibilités !

1. $a^{N-1} = 1 [N]$
2. Soit $N - 1 = 2^s d$
3. On teste
 1. Soit $\exists 0 \leq r < s$ t.q. $a^{2^r d} = -1[N]$
 2. Soit $a^d = 1[N]$

Theorem : Soit N un entier impair composé > 9 , il existe au plus $\frac{\varphi(N)}{4}$ tels a 's, qui passent le test ci-dessus.

On verra un exemple encore sur 561

Distribution des nombres premiers

- Théorème des nombres premiers
 - Soit $\pi(n)$ le nombre de nombres premiers inférieurs ou égaux à n

$$\pi(n) \sim \int_2^n \frac{dx}{\ln x} \sim \frac{n}{\ln n} + \frac{n}{(\ln n)^2} + \frac{2n}{(\ln n)^3} + \dots$$

- En pratique
 - Le nombre de premiers inférieurs à n est de l'ordre de $\frac{n}{\ln n}$
 - Il y a environ 2^{1014} nombres premiers de 1024 bits
 - La probabilité qu'un nombre x soit premier est proche de $\frac{1}{\ln n}$
 - Probabilité de 1/710 pour un entier de 1024 bits ($\ln n = \log n / \log e$)

RSA – Génération des clés

- Génération des clés (pk, sk)
 - Soit $N = p \times q$ (p et q premiers)
 - L'ordre du groupe multiplicatif $\mathbb{Z}_N^* = \varphi(N) = (p - 1)(q - 1)$
 - Soit e un entier premier avec $\varphi(N) = (p - 1)(q - 1)$
 - Soit d un entier qui satisfait $d \cdot e = 1 \pmod{\varphi(N)}$
 - Bézout $\Rightarrow d \cdot e + u \cdot \varphi(N) = 1$
- Clé publique
 - $N = pq$ module public
 - e exposant public
- Clé privée
 - $d = e^{-1} \pmod{\varphi(N)}$
 - les premiers p et q

Algorithme d'Euclide (étendu)

- Algorithme d'Euclide étendu
 - Calcul des coefficients (u, v) tels que $au + bv = d = \text{pgcd}(a, b)$
 - Théorème de Bézout
 - Définition : (u, v) sont les coefficients de Bézout pour les deux entiers naturels a et b
 - Propriétés : a et b sont premiers entre eux si et seulement s'il existe deux entiers relatifs (u, v) tels que $au + bv = 1$
- Exercice: trouver deux entiers u, v tq $120.u + 23.v = \text{pgcd}(120, 23)$

$$120 = 23 \times 5 + 5$$

$$23 = 5 \times 4 + 3$$

$$5 = 3 \times 1 + 2$$

$$3 = 2 \times 1 + 1$$

$$2 = 1 \times 2 + 0$$

Recherche des coefficients de Bézout:

$$\begin{aligned}
 1 &= 3 && - && && 2 \times 1 \\
 &= 23 - 4 \times 5 && - && && (5 - 3 \times 1) \\
 &= 23 - 4 \times 5 && - && && ((120 - 5 \times 23) - (23 - 4 \times 5)) \\
 &= 23 - 4 \times (120 - 5 \times 23) && - && && ((120 - 5 \times 23) - (23 - 4 \times (120 - 5 \times 23))) \\
 &= -9 \times 120 + 47 \times 23
 \end{aligned}$$

$$\text{pgcd}(120, 23) = 1$$

RSA – Chiffrement/Déchiffrement

- Chiffrement
 - $Enc(pk = (N, e), m) = c = m^e \pmod{N}$
- Déchiffrement
 - $Dec(sk = d, c) = c^d \pmod{N}$
- Vérification
 - Théorème d'Euler
 - $c^d = (m^e)^d = m^{1-u \cdot \varphi(N)} = m^1 \cdot (m^{-u})^{\varphi(N)} = m \pmod{N}$

Algorithme d'exponentiation x^n

- Algorithme naïf
 - Calcul de $n - 1$ multiplications successives
 - $x^n = x \cdot x \cdot x \cdots x$
- Meilleure idée : décomposer n en écriture binaire
 - $n = 16 = 2^4$
 - $x^{16} = \left(\left((x^2)^2 \right)^2 \right)^2$
 - 4 *multiplications* au lieu de $n = 2^4 - 1 = 15$
- Exemple du calcul de x^{11}
 - $11 = 8 + 2 + 1 = 2^3 + 2^1 + 2^0$
 - Calcul de $x, x^2, x^4, x^8 = \left((x^2)^2 \right)^2$
 - $x^{11} = x^8 \cdot x^2 \cdot x$

Efficacité de RSA

- Le coût est celui d'une exponentiation modulaire avec exposant e ou d
- Chiffrement : $Enc(pk = (e, N), m) = c = m^e \pmod{N}$
 - 1 exponentiation $m^e \pmod{N}$ avec un e potentiellement petit
- Déchiffrement : $Dec(sk = d, c) = c^d \pmod{N}$
 - Utilisation du Théorème des Restes Chinois (connaissant p et q)
 - 1 exponentiation $c^d \pmod{N}$
- On peut accélérer le déchiffrement par Théorème des Restes Chinois (CRT)

Théorème des Restes Chinois (CRT)

- Soit $N_1, N_2, \dots, N_k$: k nombres premiers entre eux t.q $\text{pgcd}(N_i, N_j) = 1$ si $i \neq j$
- Donné $a_1, a_2, \dots, a_k$, $\exists$ un entier x unique modulo : $N = \prod_{i=1}^k N_i$, tel que $x = a_i \pmod{N_i}$, $1 \leq i \leq k$ calculé comme suit :

$$M_i = \prod_{j \neq i} N_j ; e_i = M_i \times (M_i^{-1} \pmod{N_i}) ; x = \sum_{i=1}^k a_i e_i \pmod{N}$$

Ex : $x = 2 \pmod{3}, x = 3 \pmod{4} : x \pmod{12} ?$

$N_1 = 3, N_2 = 4 ; x = a_1 = 2 \pmod{N_1}, x = a_2 = 3 \pmod{N_2}$

$M_1 = N_2 = 4 ; M_2 = N_1 = 3 ;$

$e_1 = M_1 \times (M_1^{-1} \pmod{N_1}) = 4 ; e_2 = M_2 \times (M_2^{-1} \pmod{N_2}) = 9$

$x = a_1 e_1 + a_2 e_2 = 8 + 27 = 11 \pmod{12}$

Solution : $x = 11 \pmod{12}$

Efficacité de RSA

- Le coût est celui d'une exponentiation modulaire avec exposant e ou d
- Chiffrement : $Enc(pk = (e, N), m) = c = m^e \pmod{N}$
 - 1 exponentiation $m^e \pmod{N}$ avec un e potentiellement petit
- Déchiffrement : $Dec(sk = d, c) = c^d \pmod{N}$
 - Utilisation du Théorème des Restes Chinois (connaissant p et q)
 - 1 exp. $c^d = c^{d \pmod{p-1}} \pmod{p}$ et 1 exp. $c^d = c^{d \pmod{q-1}} \pmod{q}$
 - Récupérer $c^d \pmod{N}$ de $c^d \pmod{p}$ et $c^d \pmod{q}$ avec CRT
- Que pensez-vous de la sécurité de ce système ?

RSA – Attaques mathématiques

- Factoriser $N = pq$ et par conséquent trouver $\varphi(N)$ et puis d
- Déterminer $\varphi(N)$ directement et trouver d
- Trouver d directement (si petit)

RSA - Précautions

- Il y a de nombreuses manières de mal utiliser RSA et d'ouvrir des failles de sécurité !
- Ne jamais utiliser de valeur N trop petite
- Ne jamais utiliser d'exposant e trop petit (*t.q.* $m^e < N$)
- N'utiliser que des clés fortes ($p - 1$ et $q - 1$ ont un grand facteur premier, pour éviter l'algorithme $p-1$ de Pollard)
- Ne pas chiffrer de blocs trop courts (*t.q.* $m^e < N$ encore)
- Ne pas utiliser de e commun, plusieurs N_i différents (peut-être attaqué avec CRT, *t.q.*, $m^e < N_1 N_2 \dots N_i$)
- Ne pas utiliser de N communs à plusieurs clés
 - Donné: $c_1 = m^{e_1} \pmod{N}$; $c_2 = m^{e_2} \pmod{N}$
 - Si $\gcd(e_1, e_2) = 1$, on a s_1 et s_2 *t.q.* $e_1 s_1 + e_2 s_2 = 1$
 - Donc, $m = c_1^{s_1} c_2^{s_2} = m^{e_1 s_1 + e_2 s_2} \pmod{N}$

Sécurité de RSA

- Chiffrement homomorphe multiplicativement
 - Si je connais le chiffré c_1 du message m_1 et le chiffré c_2 du message m_2 , alors je peux calculer le chiffré du message $m_1 \cdot m_2$
 - $Enc((N, e), m_1) \cdot Enc((N, e), m_2) = c_1 \cdot c_2 = m_1^e \cdot m_2^e = (m_1 \cdot m_2)^e \pmod{N} = Enc((N, e), m_1 \cdot m_2)$
 - Attaque à chiffré choisi $\Rightarrow$ RSA non IND-CCA
- Chiffrement déterministe = si l'on chiffre plusieurs fois le même message, on obtient le même chiffré $\Rightarrow$ RSA non IND-CPA

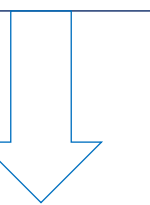
Sécurité de RSA

- RSA est considéré comme sûr
 - On ne sait pas comment déchiffrer sans connaître l'exposant d de la clé secrète
 - Trouver la clé secrète d est (pensé) équivalent à factoriser $N = pq$
 - Pas d'algorithme polynomial en temps en fonction de la taille des données (la taille des nombres N et e) pour factoriser N
 - Meilleur algorithme connu est sous-exponentiel (crible algébrique) :
$$\mathcal{O}\left(e^{1,92(\ln N)^{1/3}(\ln \ln N)^{2/3}}\right)$$
- Pour évaluer et prouver la sécurité du RSA
 - Evaluer la rapidité des algorithmes de factorisations de grands nombres entiers
 - Démontrer que la clé secrète de déchiffrement d ne peut pas être obtenue sans factoriser $N = pq$
 - Montrer que on ne peut pas déchiffrer un message sans la clé secrète d

Problème RSA

- Problème RSA : extraction de racine e -ième
 - $x \rightarrow x^e \pmod{N}$ facile
 - $y = x^e \pmod{N} \rightarrow x$ difficile, excepté avec la trappe $d = e^{-1} \pmod{\varphi(N)}$
- Réduction
 - Si on connaît la factorisation, on casse RSA : RSA se réduit à la factorisation !
 - Le contraire est peut-être faux : calculer des racines e -ièmes sans factoriser ?
- En pratique : la factorisation est la seule méthode connue pour casser RSA

RSA



FACT

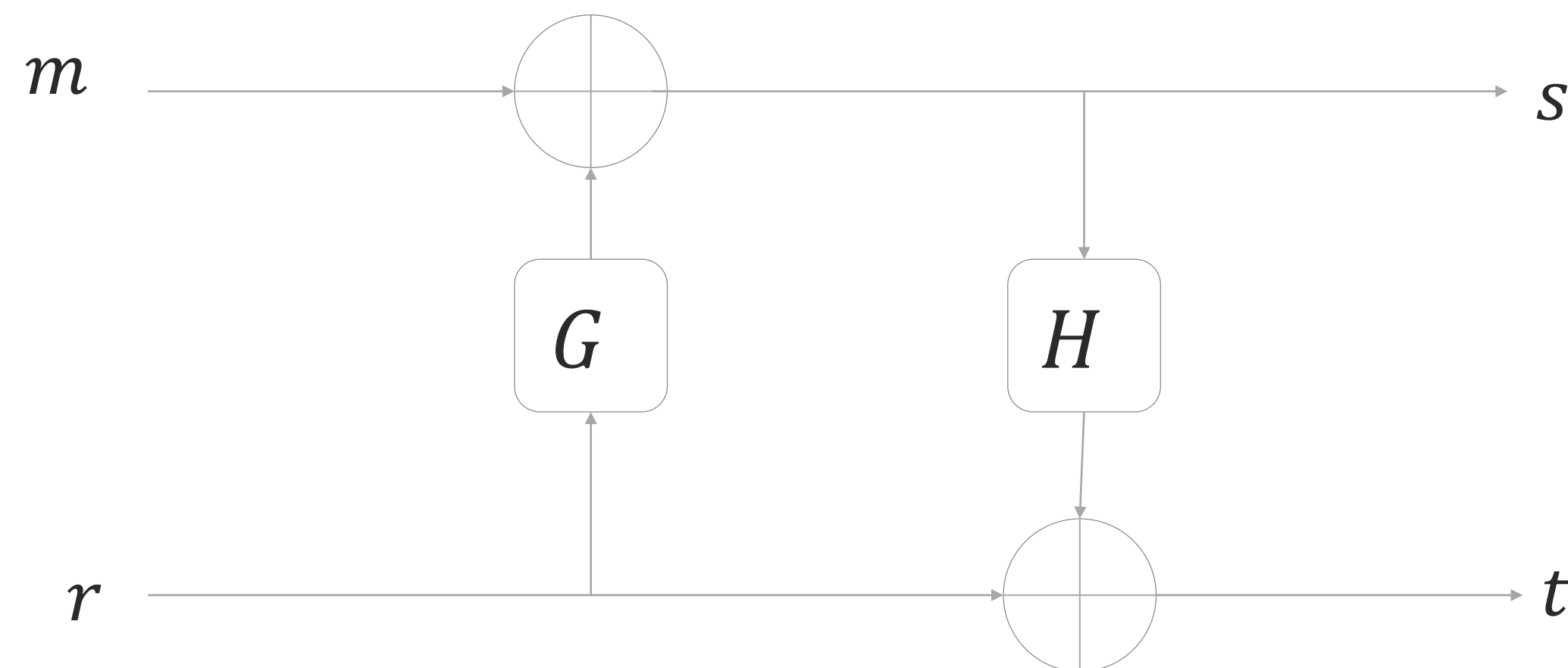
Cryptographie basé sur RSA

Usage autorisé	Jusqu'en 2021	Après 2021
Si la taille minimale du module (RSA) est	2048 bits	3072 bits

- Peut-on obtenir mieux que OW-CPA ?

RSA-OAEP (Bellare et Rogaway, 1994)

- OAEP = Optimal Asymmetric Encryption Padding
- Soient G et H deux fonctions de hachage et soit r un aléa



$$c = RSA((N, e), s || t)$$

- Comment déchiffrer ?
- A quel schéma cela vous fait-il penser ?

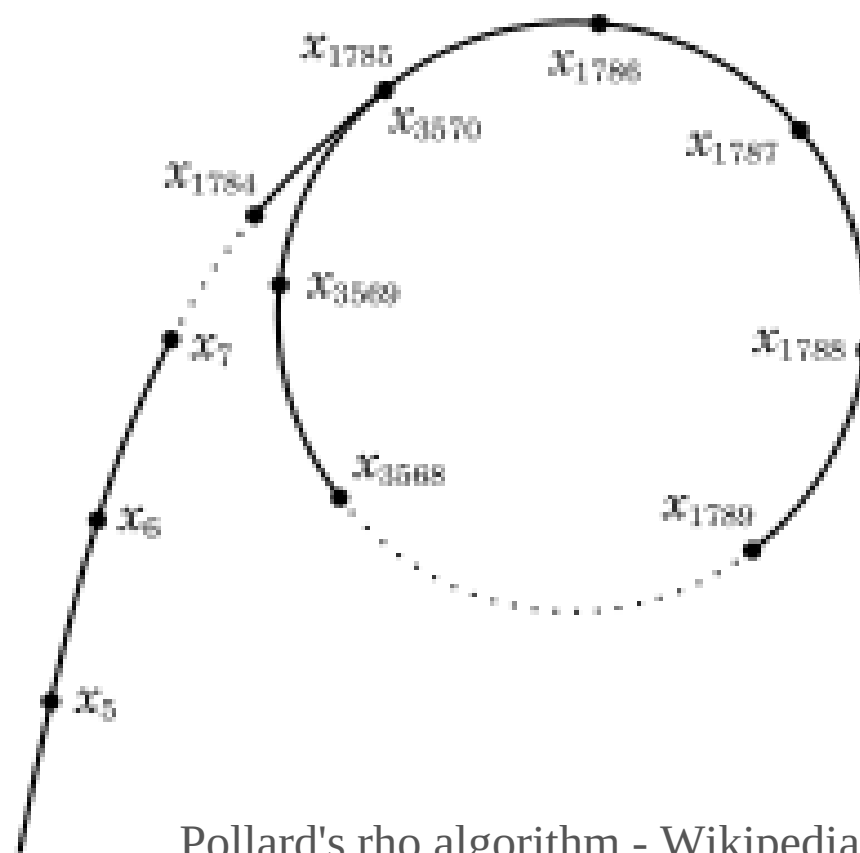
Sécurité de RSA-OAEP

- OAEP est une solution générique et peut s'appliquer à n'importe quel schéma
 - On obtient un **schéma IND-CPA** si la fonction utilisée est à sens unique à trappe (c'est le cas de **RSA**)
- Peut-on obtenir IND-CCA1 ?
 - Si la fonction utilisée est à sens unique sur un domaine partiel, à trappe, alors on obtient IND-CCA1
 - $(a, b) \rightarrow f(a||b)$ à sens unique, à trappe
 - $f(a||b) \rightarrow a$ aussi à sens unique
 - Vrai pour RSA sous l'hypothèse RSA
- Quid de IND-CCA2 ?
 - Génériquement, on ne sait pas
 - Pour RSA, on sait que c'est vrai $\Rightarrow$ Preuve par réduction
 - Mais la réduction n'est pas bonne $\Rightarrow$ multiplier par deux la taille du module $\Rightarrow$ 6144 ($=3072 \times 2$) bits !

Logarithme discret et chiffrement ElGamal

Calcul d'un logarithme discret

- Problème
 - Soit $(\mathbb{G}, \cdot)$ un groupe d'ordre N
 - $(\mathbb{G}, N, g, x) \rightarrow y = g^x$ est facile
 - $(\mathbb{G}, N, g, y) \rightarrow x \in [0, N - 1]$ difficile
- Algorithmes de calcul d'un logarithme discret
 - Recherche exhaustive $\Rightarrow \mathcal{O}(N)$
 - [Baby-Step-Giant-Step](#) $\Rightarrow \mathcal{O}(\sqrt{N})$ éléments de groupe à stocker, $\mathcal{O}(\sqrt{N})$ multiplications de groupe
 - [Pollard's rho](#) $\Rightarrow \mathcal{O}(\sqrt{N})$ opérations de groupe (**qui fonctionne également pour les courbes elliptiques**)



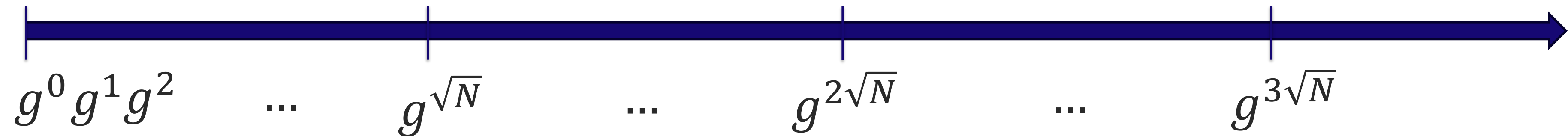
Pollard's rho algorithm - Wikipedia

Cryptographie basé sur le logarithme discret

Usage autorisé	Jusqu'en 2021	Après 2021
Si la taille minimale du module premier du groupe modulaire est	2048 bits	3072 bits (=AES 128)
Si la taille minimale du module premier du groupe (sur courbe elliptique) est	256 bits	256 bits (=AES 128)

- [Crible algébrique](#) $\Rightarrow 2^{\mathcal{O}((\log N)^{1/3})}$ opérations de groupe

Logarithme discret – Baby-Step-Giant-Step



1. On calcul une liste de $\sqrt{N}$ valeurs : $\{g^0, g^1, g^2, \dots, g^{\sqrt{N}-1}\}$.
2. Soit $y = g^x$, on considère la boucle suivante :
 1. Si y est dans la liste (recherche avec un tableau trié ou une table de hachage), on s'arrête.
 2. Sinon, on calcule $y = y / g^{\sqrt{N}}$, continuer à l'étape 1

Complexité :

On note par a le nombre de passages à l'étape 2, et $y = g^b$ à l'étape 1. Alors, on sait que $x = a\sqrt{N} + b$.

Il nous faut un espace de stockage de $O(\sqrt{N})$ et un temps d'exécution de $\sqrt{N} \times v$ avec v comme temps d'une recherche unique.

Système de chiffrement ElGamal (1985)

- Génération des clés
 - Choisir un groupe $(\mathbb{G}, \cdot)$ d'ordre N
 - Choisir la clé secrète $sk = s \in \mathbb{Z}_N$
 - Calculer la clé publique $pk = y = g^s$
- Instances possibles
 - Groupe multiplicatif $\mathbb{Z}_p^*$ des entiers modulo un nombre premier p
 - Groupe des éléments inversibles $\mathbb{Z}_N^*$ pour un entier N composé

Fonction à sens-unique à trappe !

- $x \rightarrow (x \cdot y^r, g^r)$ facile
- $(x \cdot y^r, g^r) \rightarrow x$ difficile, excepté avec la trappe $s = \log_g y$ ($\frac{x \cdot y^r}{(g^r)^s} = \frac{x \cdot g^{rs}}{(g^r)^s} = x$)

- Chiffrement du message $m \in \mathbb{G}$
 - Choisir un aléa $r \in \mathbb{Z}_N$
 - Calculer $T_1 = m \cdot y^r$ et $T_2 = g^r$
- Déchiffrement du chiffré (T_1, T_2)
 - Calculer $m = T_1 / T_2^s$
- Vérification
 - $\frac{T_1}{T_2^s} = \frac{m \cdot y^r}{(g^r)^s} = \frac{m \cdot g^{rs}}{g^{rs}} = m$

Sécurité d'ElGamal

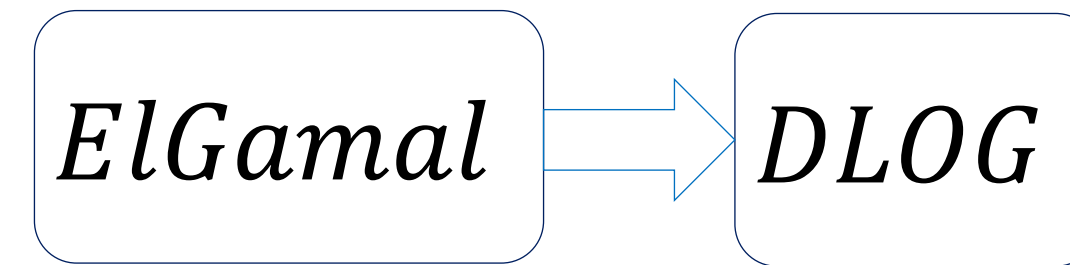
- Chiffrement homomorphe multiplicativement
 - Si je connais le chiffré $c = (T_1, T_2)$ du message m et le chiffré $c' = (T_1', T_2')$ du message m' , alors je peux calculer le chiffré du message $m \cdot m'$
 - $D_1 = T_1 \cdot T_1'$ et $D_2 = T_2 \cdot T_2'$
 - Attaque à chiffré choisi $\Rightarrow$ ElGamal non IND-CCA
- ElGamal avec le même aléa
 - Ne jamais utiliser deux fois le même aléa !
 - Si m et m' sont chiffrés avec le même aléa r , alors $c = (T_1, T_2) = (my^r, g^r)$ et $c' = (T_1', T_2') = (m'y^r, g^r)$, d'où

$$\frac{m}{m'} = \frac{T_1}{T_1'}$$

Sécurité d'ElGamal

- Réduction

- La recherche de la clé privée s à partir de la clé publique $y = g^s$ est équivalente au problème du logarithme discret
- ElGamal se réduit au logarithme discret !



- En pratique

- Si le problème du logarithme est résolu efficacement, alors ElGamal sera cassé
- Le contraire est peut-être faux !
- Rien ne prouve qu'il ne soit pas cassable par un autre moyen
- Le log discret est la seule méthode connue pour casser ElGamal

Modèle de sécurité

- La sécurité parfaite au sens de Shannon est *impossible* en clé publique
 - La connaissance de la clé publique et du chiffré biaise la distribution de probabilité du message clair (qui est en fait *unique*)
 - Quel espoir : un attaquant avec une capacité de calcul polynomiale peut-il exploiter ce biais et accéder à une information
- **Modèle de sécurité**
 - Spécifier **les buts** et **les moyens** (calculs, accès aux ressources...) de l'attaquant
 - Examiner quelles sont **les chances** pour un attaquant d'atteindre son but avec les moyens spécifiés
 - **Probabilité de succès**
 - **Preuve de sécurité**
 - **Conclure** (pertinence du modèle, choix des paramètres, . . .)

L'attaquant/adversaire

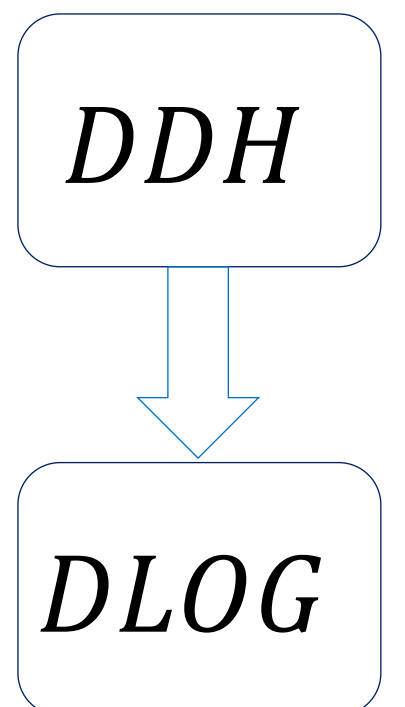
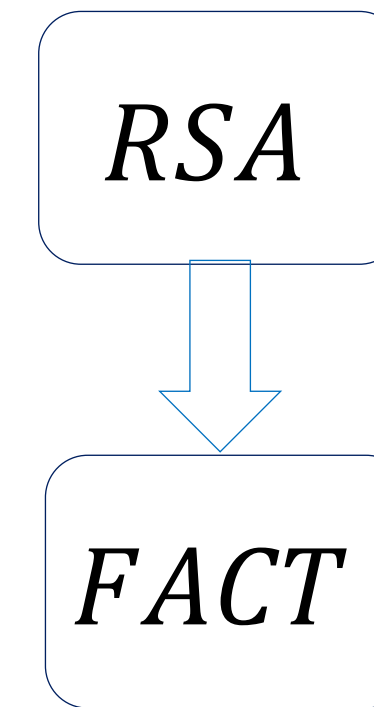
- L'attaquant
 - **Le plus intelligent possible** : il peut faire toutes les opérations qu'il souhaite
 - Il dispose d'un temps limité : on ne considère pas les attaques faisables en 2^{60} ans
- **Modélisation** : machine de Turing
 - Probabiliste
 - Polynomiale (circuit) en la taille des clés
 - Il a un temps raisonnable d'exécution
- Deux approches pour évaluer sécurité
 - **Cryptanalyse** : exhiber un attaquant qui est victorieux
 - **Réduction** : preuve de sécurité quel que soit l'attaquant, qui est alors une boîte noire

Preuve de sécurité – preuve par réduction

- Hypothèse
 - Un problème P difficile = il n'y a pas d'algorithme polynomial
 - $P = FACT, RSA, DLOG, DDH, \dots$
- Réduction
 - si $\mathcal{A}$ un adversaire (polynomial) casse le schéma de chiffrement
 - alors $\mathcal{A}$ peut être utilisé pour résoudre P en temps polynomial (ce qui est considéré impossible)
- Résultat de sécurité : il n'existe pas d'adversaire polynomial

Hypothèses de sécurité

- Problème de la factorisation (*FACT*)
 - Etant donné $N = p \times q$, trouver p et q
- Problème *RSA*
 - Etant donné $N = p \times q, e, y = x^e \pmod{N}$, trouver x
- Problème du logarithme discret (*DLOG*)
 - Etant donné $(\mathbb{G}, N, g, y = g^x)$, trouver x
- Problème Décisionnel Diffie Hellman (*DDH*)
 - Etant donnés $\mathbb{G}$ (ordre N), g, g^a, g^b, g^c ($c \stackrel{\$}{\leftarrow} \mathbb{Z}_N$) décider si $g^c = g^{ab}$
 - C'est-à-dire, étant donné (g, g^a, g^b) , g^{ab} est supposé caché



$$a \stackrel{\$}{\leftarrow} \mathbb{Z}_N$$

$$g^a$$



$$b \stackrel{\$}{\leftarrow} \mathbb{Z}_N$$

$$g^b$$

$$k = (g^b)^a = g^{ab}$$

$$k = (g^a)^b = g^{ab}$$

Preuve de sécurité (CPA) de chiffrement ElGamal

DDH

ElGamal

- **Monde 1:**

- Génération des clés

- Choisir un groupe $(\mathbb{G}, \cdot)$ d'ordre N
- Choisir la clé secrète $sk = s \in \mathbb{Z}_N$
- Calculer la clé publique $pk = y = g^s$

- Chiffrement du message $m \in \mathbb{G}$

- Choisir un aléa $r \in \mathbb{Z}_N$
- Calculer $T_1 = m \cdot y^r$ et $T_2 = g^r$

- **Monde 2:**

- Génération des clés

- Choisir un groupe $(\mathbb{G}, \cdot)$ d'ordre N
- Choisir la clé secrète $sk = x \in \mathbb{Z}_N$
- Calculer la clé publique $pk = y = g^s$

- Chiffrement du message $m \in \mathbb{G}$

- Choisir un aléa $z \in \mathbb{Z}_N$
- Calculer $T_1 = m \cdot g^z$ et $T_2 = g^r$

- **Monde 1 :** $pk = y = g^s$, $T_1 = m \cdot y^r = m \cdot g^{sr}$ et $T_2 = g^r$

- **Monde 2 :** $pk = y = g^s$, $T_1 = m \cdot y^r = m \cdot g^z$ et $T_2 = g^r$, $z \xleftarrow{\$} \mathbb{Z}_N$

- $\Rightarrow$ Monde 1 et 2 sont indistinguable sur *DDH*

DDH

DDH

- $(g^s, g^r, m_0 \cdot g^{sr}) \approx (g^s, g^r, m_0 \cdot g^z) \approx (g^s, g^r, m_1 \cdot g^z) \approx (g^s, g^r, m_1 \cdot g^{sr})$

- $\Rightarrow \text{Enc}(pk, m_0)$ est indistinguable de $\text{Enc}(pk, m_1)$

Signature numérique – objectif et propriétés

- Objectif : reproduire les caractéristiques d'une signature manuscrite
 - Lier un document à son auteur
 - Rendre la signature difficilement imitable
 - Responsabilité de l'auteur (juridique)
- Propriétés
 - La signature numérique dépend du signataire et du document
 - La signature appartient à un seul document
 - Impossible à découper une signature sur un message et la recoller sur un autre
 - Le document signé ne peut pas être modifié
 - La signature ne peut pas être falsifiée
 - La signature ne peut pas être reniée

RSA et Signature RSA

Signature RSA – Génération des clés

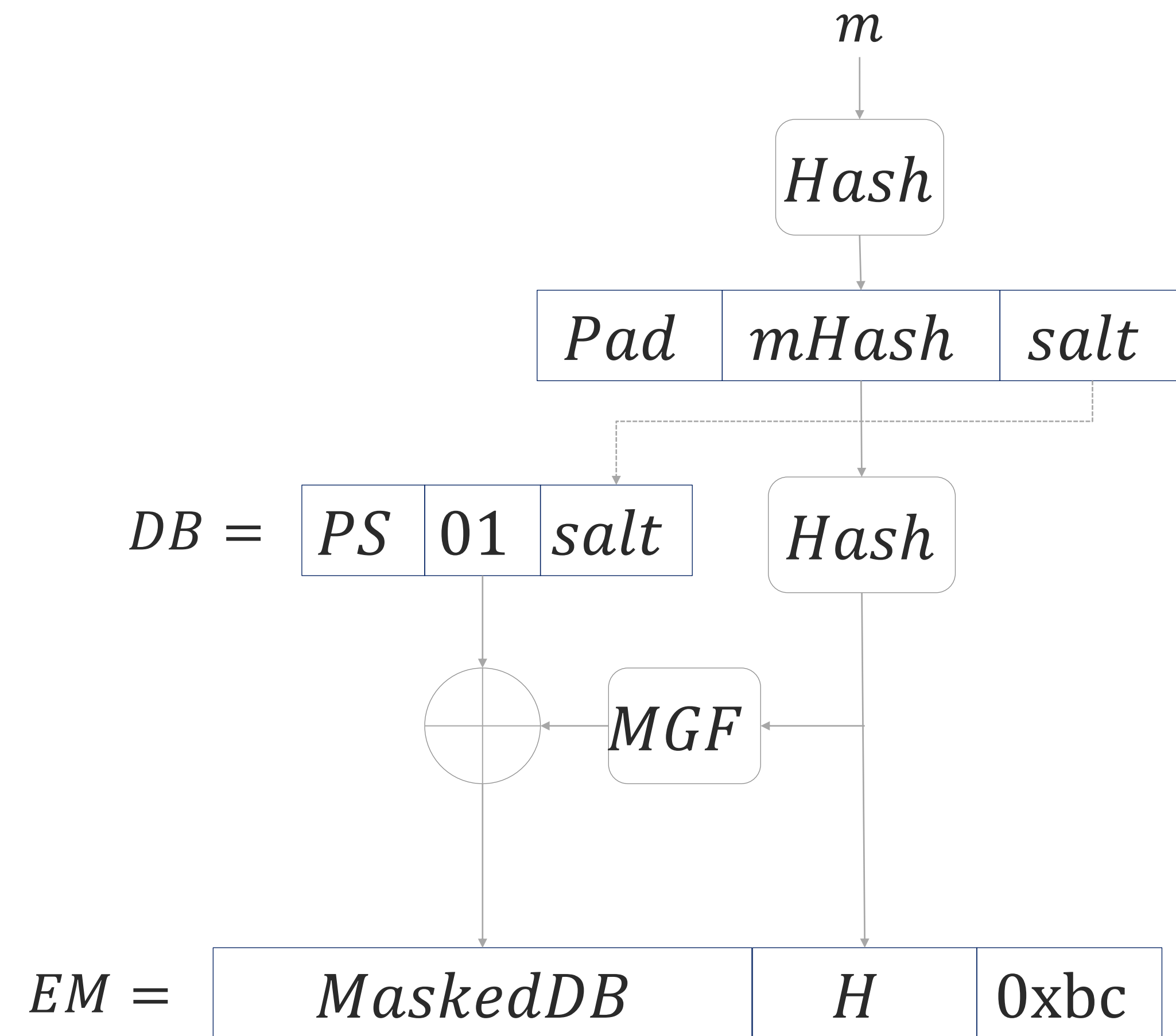
- Identique au chiffrement
- Génération des clés (pk, sk)
 - Soit $N = p \times q$ (p et q premiers)
 - L'ordre du groupe multiplicatif $\mathbb{Z}_n^* = \varphi(N) = (p - 1)(q - 1)$
 - Soit e un entier premier avec $\varphi(N) = (p - 1)(q - 1)$
 - Soit d un entier qui satisfait $d \cdot e = 1 \pmod{\varphi(N)}$
 - Bézout $\Rightarrow d \cdot e + u \cdot \varphi(N) = 1$
- Clé publique
 - $N = pq$ module public
 - e exposant public pour la vérification
- Clé privée
 - $d = e^{-1} \pmod{\varphi(N)}$ pour la signature
 - les premiers p et q

RSA – Signature/Vérification

- Signature
 - $Sig(sk = d, m) = \sigma = m^d \pmod{N}$
- Vérification
 - $Ver(pk = (N, e), \sigma, m)$: vérifier si $m = \sigma^e \pmod{N}$
- Attention !
 - Vérifier une signature ne veut pas systématiquement dire « déchiffrer le message »
- Inconvénients
 - Signature déterministe : on peut la réutiliser pour envoyer de nouveau le même message signé
 - Malléabilité : à partir de deux messages signés (σ_1, m_1) et (σ_2, m_2) on crée un nouveau message signé $(\sigma_1 \sigma_2 = (m_1 m_2)^d, m_1 m_2)$

RSA-PSS (Bellare and Rogaway, 2017)

- PSS = **Probabilistic** Signature Scheme
- Idée : mécanisme de « hash-and-sign »
- Nécessite
 - Fonction de hashage $Hash$
 - Fonction de génération de masque MGF (supporte sorties de tailles différentes)
 - $salt$ = chaîne de caractères aléatoires
 - Pad = 8 octets à 0x00
 - PS = 1 octet à 0x00
- Schéma infalsifiable face à un adversaire ayant accès à un oracle de signature (sûr au sens UF-CMA)

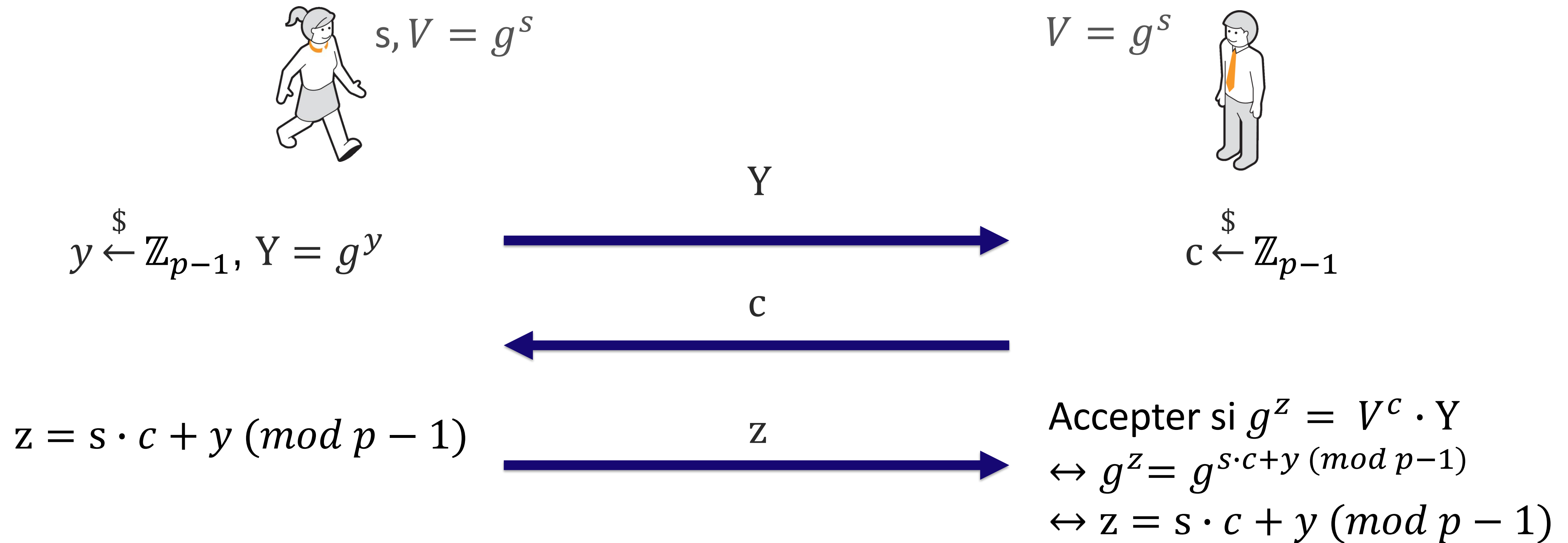


Logarithme discret et Signature Schnorr

Signature Schnorr (Claus Schnorr, 1990)

- Paramètres
 - p : grand nombre premier
 - g : génératrice du groupe $\mathbb{Z}_p^*$
- Génération de clés
 - Clé secrète : $s \in \mathbb{Z}_{p-1}$
 - Clé publique : $(p, g, V) \mid V = g^s \bmod p$
- Signature
 - Choisir un nombre aléatoire $y \in \mathbb{Z}_{p-1}$
 - Calculer $Y = g^y \bmod p$
 - Calculer $c = H(Y||m)$
 - Calculer $z = s \cdot c + y \bmod p - 1$
 - Signature du message m : (Y, c, z)
- Vérification
 - Verifier si $c = H(Y||m)$ et $g^z/V^c = Y = g^y \bmod p$
 - $g^z/V^c = g^y \bmod p \leftrightarrow g^z = g^{s \cdot c + y} \bmod p \leftrightarrow z = s \cdot c + y \bmod p - 1$

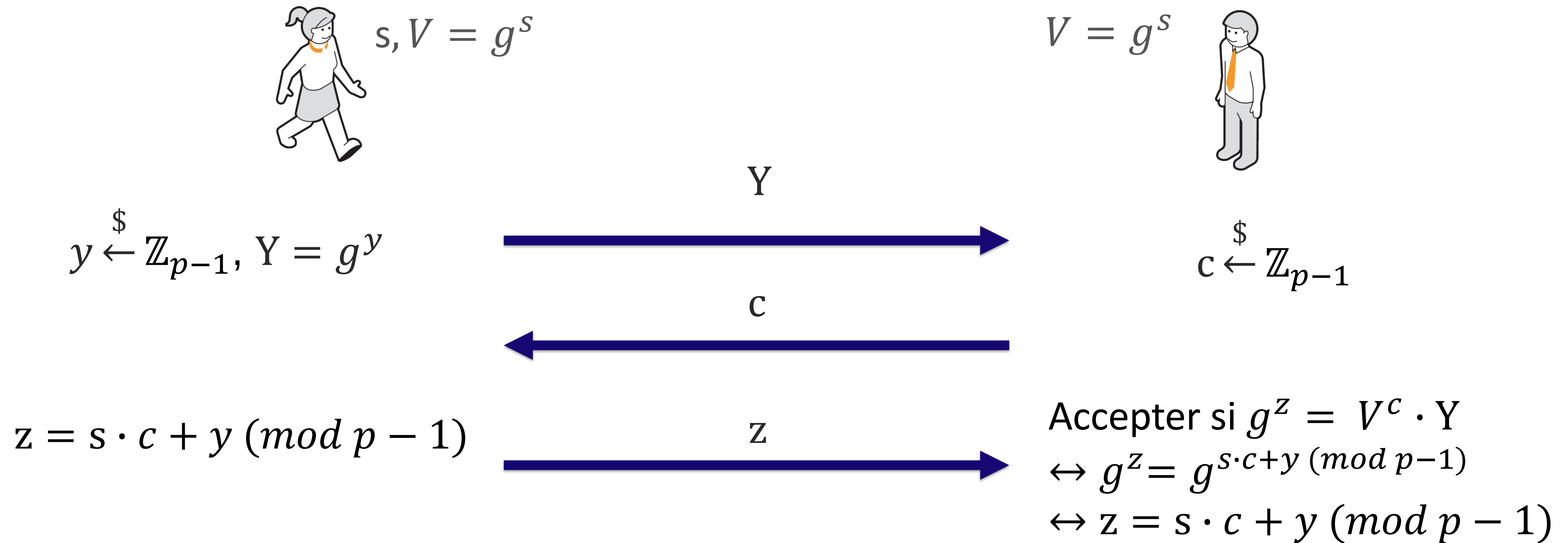
Preuve de connaissance sous-jacent signature Schnorr



Preuve de connaissance de s :

- Lancer Alice (avec challenge V) deux fois avec challenge c_1 et c_2 , obtenir z_1 et z_2 tel que : $z_1 = s \cdot c_1 + y$ et $z_2 = s \cdot c_2 + y$
- On peut calculer $s = (z_1 - z_2) / (c_1 - c_2) = (s \cdot c_1 + y - s \cdot c_2 - y) / (c_1 - c_2)$

Preuve de connaissance sous-jacent signature Schnorr

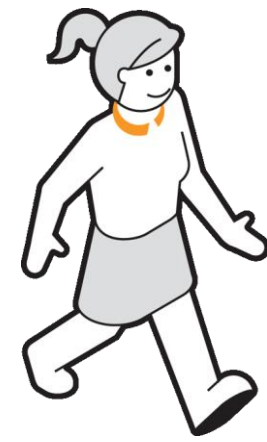


Sans divulguer la connaissance de s :

- On peut simuler la transcript (Y, c, z) sans connaître s :

- Choisit $c \xleftarrow{\$} \mathbb{Z}_{p-1}$; Choisit $z \xleftarrow{\$} \mathbb{Z}_{p-1}$
- Calcule $Y = g^z / V^c$

Preuve de connaissance $\Rightarrow$ signature Schnorr



$$s, V = g^s$$

$$V = g^s$$



$$y \xleftarrow{\$} \mathbb{Z}_{p-1}, Y = g^y$$

$$\cancel{c \xleftarrow{\$} \mathbb{Z}_{p-1}}$$

$$c = H(Y || m)$$

$$z = s \cdot c + y \pmod{p-1}$$

$$\sigma = (Y, c, z)$$

Accepter si $g^z = V^c \cdot Y$
 $\Leftrightarrow g^z = g^{s \cdot c + y \pmod{p-1}}$
 $\Leftrightarrow z = s \cdot c + y \pmod{p-1}$

Transformation de Fiat-Shamir : de interactive à non-interactive

UF-CMA de la signature Schnorr

Adversaire

$O^{Sign}(\{M_i\}_i)$

$\{\sigma_i\}_i$

(M^*, σ^*)

Challenger

Simule σ_i sans s

$M^* \notin \{M_i\}_i$

Si σ^* est valide sur M^* ,
Adv réussit

Si $\Pr[\text{Adv réussit}]$ est *non* – $\text{negl}(\lambda)$, on va lancer **Adversaire** deux fois et obtenir (M_1^*, σ_1^*) et (M_2^*, σ_2^*) pour extraire s .

Cryptographie sur les courbes elliptiques (qui forme un autre groupe commutatif)

Courbes elliptiques

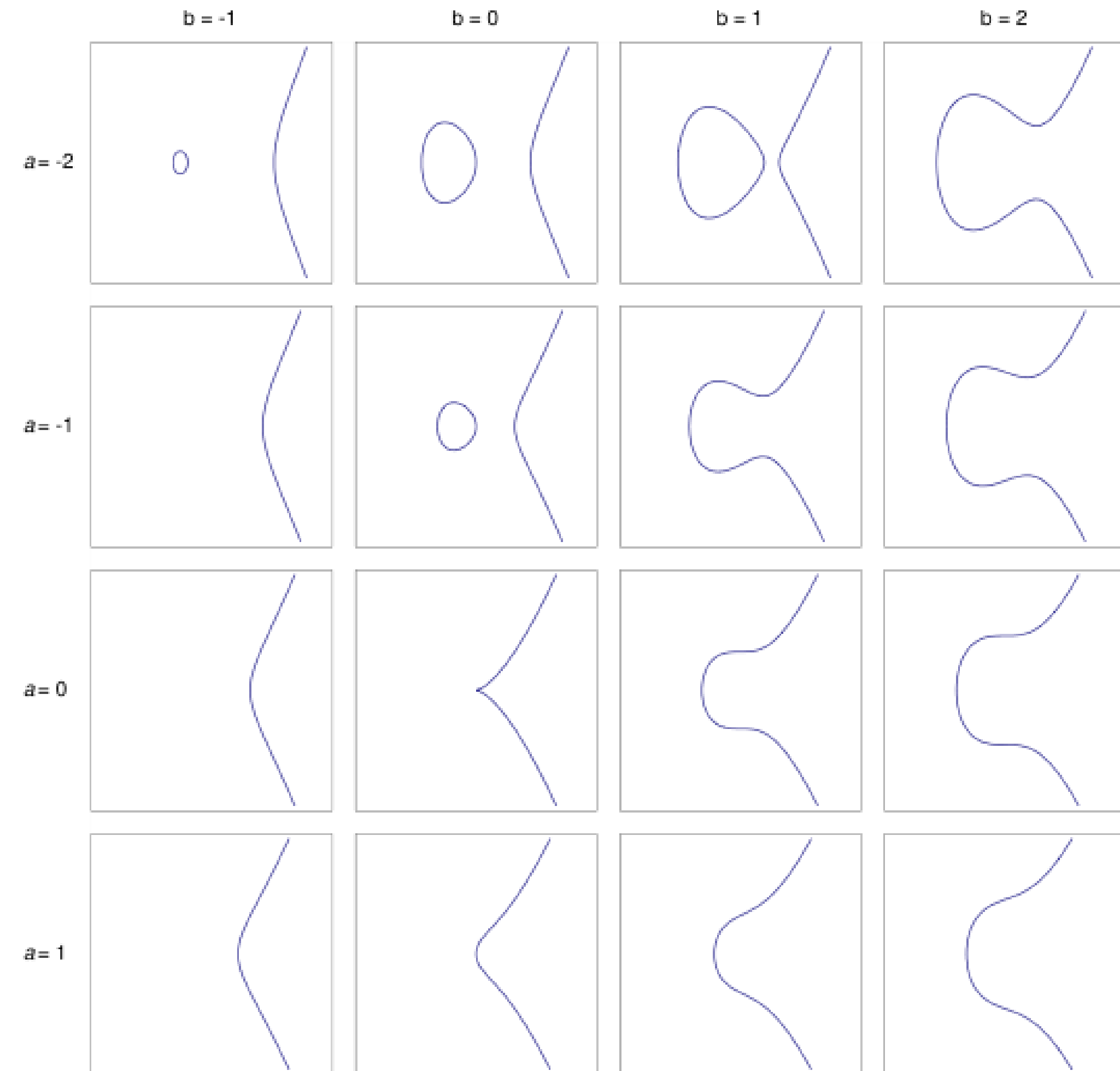
- Courbe elliptique : courbe algébrique peut être représentée dans un plan (nombres réels) par une équation (de Weierstrass) :

$$y^2 = x^3 + ax + b$$

- Courbe sans singularités (ou non-singulière) : si le discriminant :

$$\Delta = -16(4a^3 + 27b^2) \neq 0$$

- Les points d'une courbe elliptique forment un **groupe**
 - Opérateur du groupe : **addition de points**



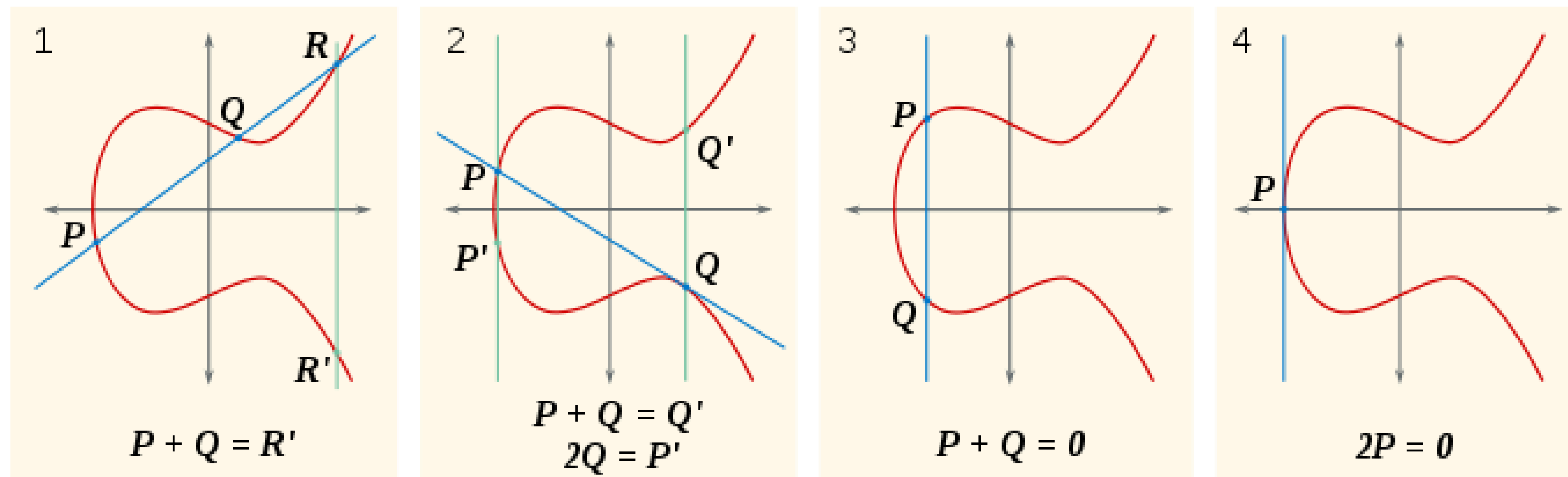
Wikipedia.org

Courbes elliptiques : addition

1. Cas ordinaire : $P + Q = R'$: point symétrique p/p à l'axe des abscisses de point d'intersection de la droite $(P Q)$ définie par P et Q et la courbe
 2. Si la droite $(P Q)$ est la tangente à la courbe en Q , le point d'intersection de $(P Q)$ et la courbe est Q donc $P + Q = Q'$: symétrique de Q
 3. Si la droite $(P Q)$ est verticale, $P + Q = 0$: point à l'infini (noté « O » ou « O_E »)
 4. Si $P = Q$, $2P$: point symétrique au point d'intersection de la tangente de la courbe au point P
- Propriétés : Soit P , $P + O = P$ (**Élément neutre**) ; il existe Q t.q. $P + Q = O$ (**Symétrique**)

Loi de
composition interne

Associativité



Graduate Texts
in Mathematics

Joseph H. Silverman
**The Arithmetic
of Elliptic Curves**

2nd Edition



Courbes elliptiques : addition (suite)

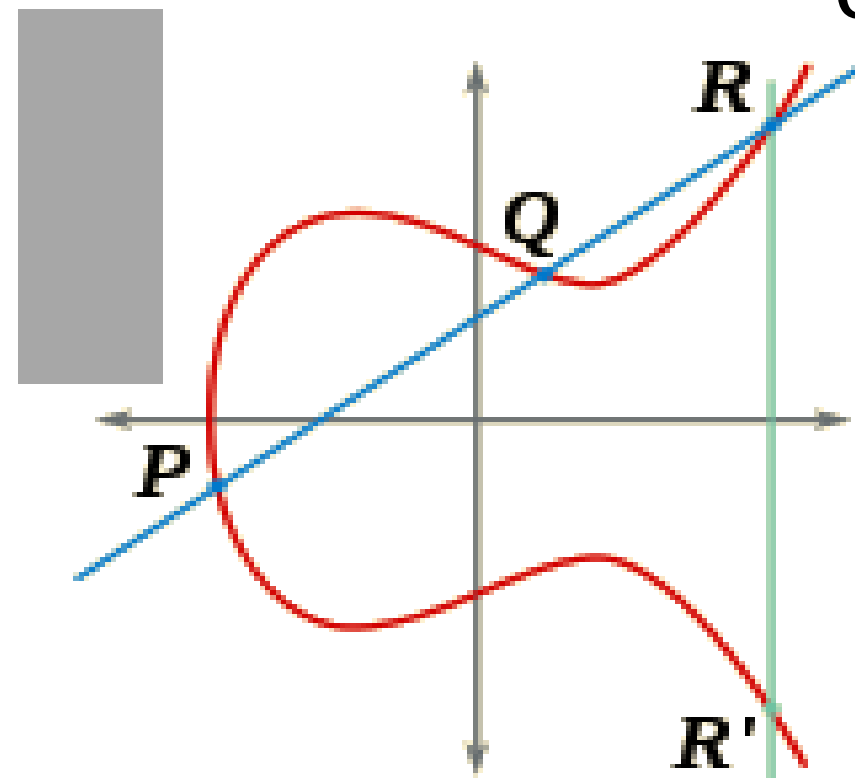
- Deux points : $P (x_P, y_P)$ et $Q (x_Q, y_Q)$

1. Si $x_P \neq x_Q$: ($y = sx + t$ est la droite (P Q))

$$s = (y_P - y_Q)/(x_P - x_Q) \text{ et } t = (y_P x_Q - y_Q x_P)/(x_Q - x_P)$$

on peut calculer (Formule de Vieta) : $x_{P+Q} = s^2 - x_P - x_Q$

$$\text{et } y_{P+Q} = -(s(x_{P+Q} - x_P) + y_P)$$



$$P + Q = R'$$

2. Si $x_P = x_Q$ et $y_P \neq y_Q$ ($y_P = -y_Q$): $P + Q = O$ (Q est l'inverse de P)

3. Si $x_P = x_Q$ et $y_P = y_Q \neq 0$:

$$x_{2P} = s^2 - 2x_P \text{ et } y_{2P} = -y_P + s(x_P - x_{2P}) ; \quad s = (3x_P^2 + a)/2y_P$$

Courbe elliptique sur un corps fini : groupe

- Une courbe elliptique peut être définie sur un corps fini :
 - Exemple : F_p corps des congruences modulo p (p : nombre premier)
 - Courbe elliptique sur F_p :

$$y^2 = x^3 + ax + b \bmod p$$

- **Théorème** : l'ensemble de points définis par la courbe elliptique sur un corps fini incluant le point à l'infini forme un groupe commutatif
 - Commutativité par rapport à l'opération d'addition de points ($P+Q = Q+P$)

Example

- Courbe elliptique E sur F_7 :

$$y^2 = x^3 + x + 6 \pmod{7}$$

$$(\Delta = -16(4 + 27 \times 36) \neq 0)$$

- Chercher les points de E?

$$E = \{(1, 1), (1, 6), (2, 3), (2, 4), (3, 1), (3, 6), (4, 2), (4, 5), (6, 2), (6, 5)\}$$

x	y^2	y_1	y_2
0	6	-	-
1	1	1	6
2	2	4	3
3	1	1	6
4	4	2	5
5	3	-	-
6	4	2	5

Cryptographie sur les courbes elliptiques

- Utilisation des courbes elliptiques en cryptographie
 - Neal Koblitz et Victor Miller en 1985
 - ECC (acronyme anglais d'“Elliptic curve cryptography”)
- Plusieurs algorithmes cryptographiques asymétriques ont été adaptés à ECC (ex: échange de clé Diffie-Hellman, signature numérique, RSA)
- ECC procure un niveau de sécurité en général supérieure aux autres algorithmes
 - Les opérations de chiffrement et de déchiffrement ont une plus grande complexité que pour d'autres algorithmes

ECC : ECDLP

- Sécurité d'ECC repose sur le problème du logarithme discret dans le groupe de la courbe elliptique (**ECDLP**)
- Problème :
 - On considère une courbe elliptique E sur un corps fini,
 - **Soit point $P \in E$, trouver l'entier r connaissant $r.P$**

La difficulté de ce problème est déterminée par la taille de la courbe elliptique

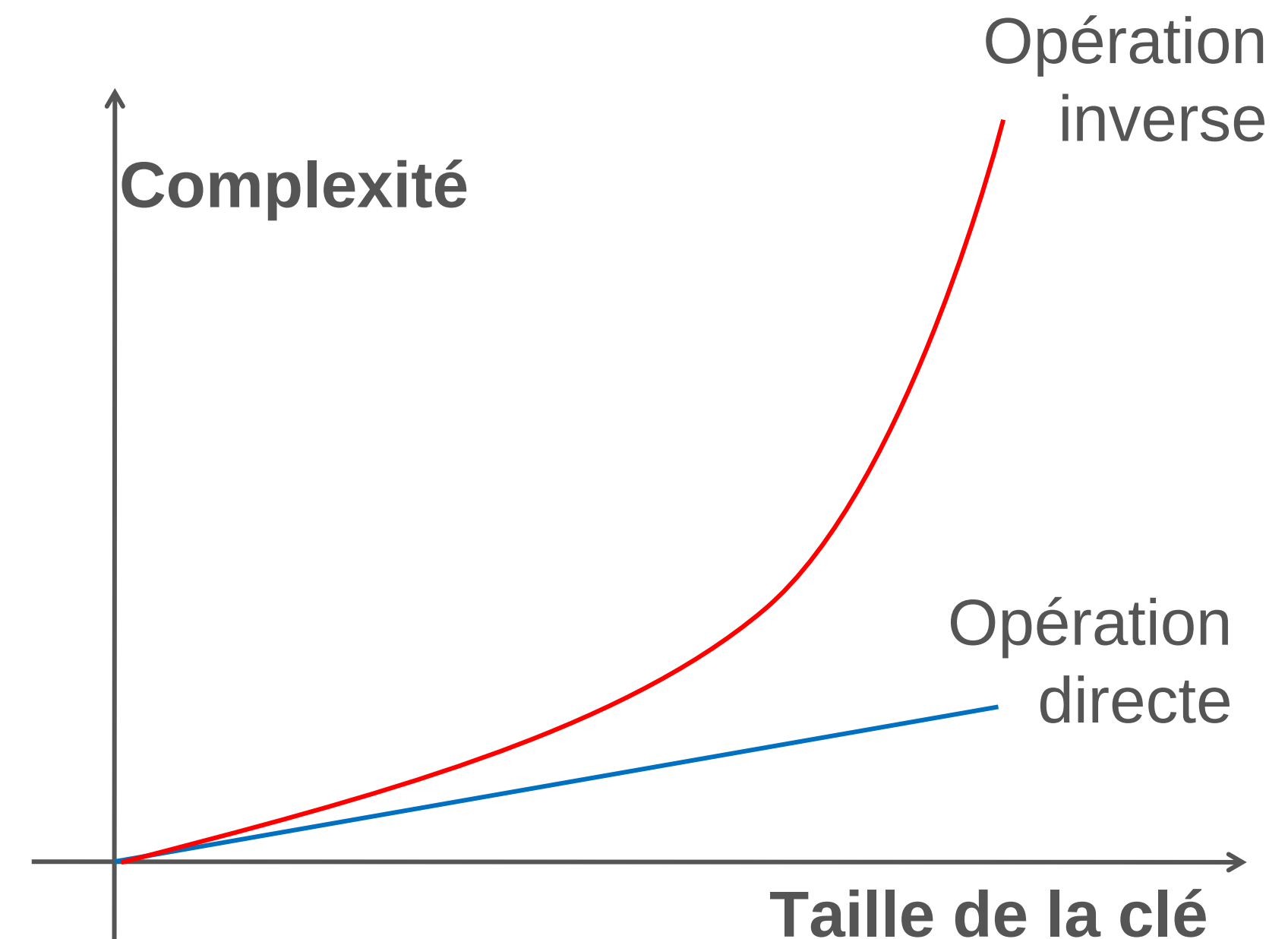
ECC : clé

- Clés d'ECC sont plus courtes qu'avec RSA
- NSA ("National Security Agency") a approuvé l'utilisation d'ECC pour protéger des documents classifiés jusqu'au niveau «Top secret » avec une clé de 384 bits

Équivalence de taille de clé (clés en bits)

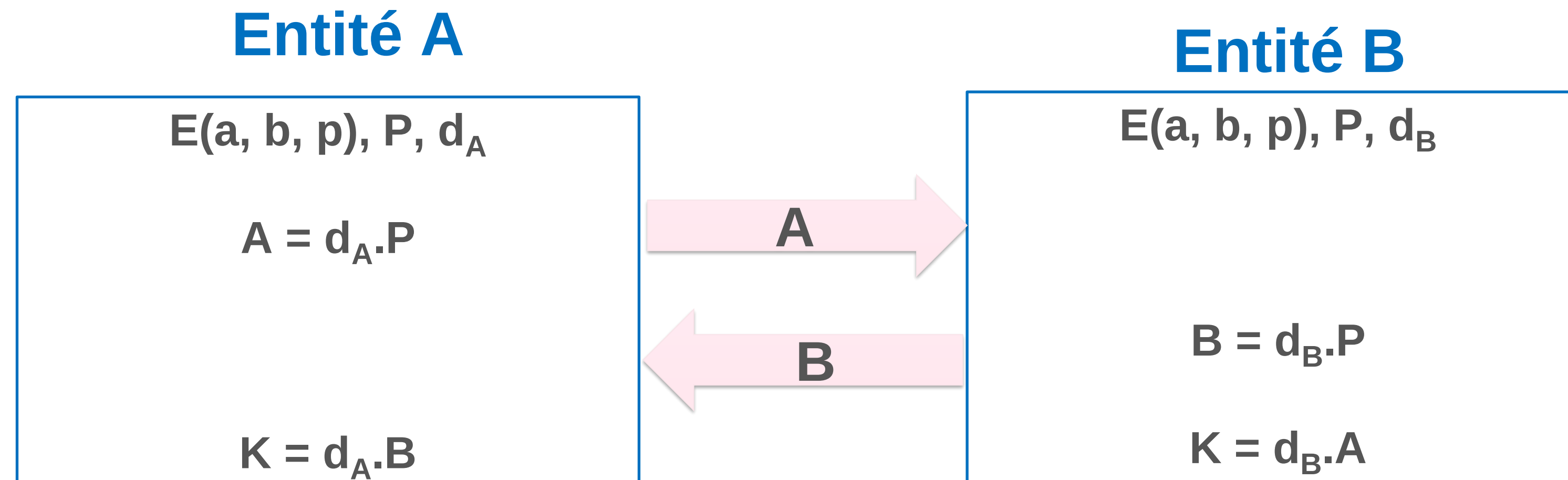
ECC	RSA	Ratio	AES
163	1024	1:6	--
256	3072	1:12	128
384	7680	1:20	192
512	15360	1:30	256

National Institute of Standards and Technology (NIST)



Échange de clé basé sur ECC

- Les deux entités choisissent publiquement une courbe elliptique $E(a, b, p)$ et un point P (qui doit être un générateur) $\in E(a, b, p)$



- Clé secrète : $K = d_A \cdot B = d_A(d_B P) = (d_A d_B)P \in E(a, b, p)$
- ECDDH $\Rightarrow$ un adversaire ne peut pas déduire K connaissant A et B

Application en ligne

Visionneuse de certificats : *.google.com

Général Détails

Hiérarchie des certificats

- ▼ GTS Root R1
 - ▼ WR2
 - *.google.com

Champs de certificat

- Pas avant
- Pas après
- Sujet
- ▼ Informations sur la clé publique du sujet
 - Algorithme de clé publique du sujet
 - Clé publique du sujet
- ▼ Extensions
 - Utilisation clé du certificat

Valeur de champ

Clé publique à courbe elliptique

Exporter...



On aura une séance d'une heure de questions-réponses avant le prochain TD.

Clé publique à courbe elliptique